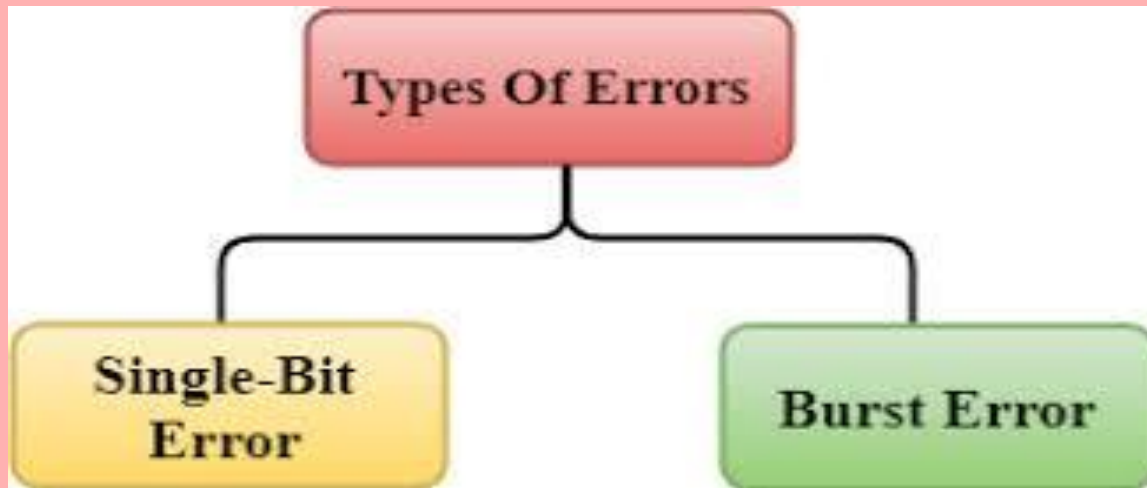


Error Detection and Correction

Basic Concept

- Networks must be able to transfer data from one device to another with complete accuracy. [?] Data can be corrupted during transmission. [?] For reliable communication, errors must be detected and corrected. [?] Error detection and correction are implemented either at the data link layer or the transport layer of the OSI model

Types of Errors



Single-bit error



In a frame, there is only one bit, anywhere though, which is corrupt.

Single-bit error

- Single bit errors are the least likely type of errors in serial data transmission because the noise must have a very short duration which is very rare. However this kind of errors can happen in parallel transmission.
- **Example:** If data is sent at 1Mbps then each bit lasts only $1/1,000,000$ sec. or $1\ \mu\text{s}$. For a single-bit error to occur, the noise must have a duration of only $1\ \mu\text{s}$, which is very rare

Burst Error



Frame contains more than 1 consecutive bits corrupted.

Burst Error

- Burst error is most likely to happen in serial transmission since the duration of noise is normally longer than the duration of a bit.
 - The number of bits affected depends on the data rate and duration of noise.
- **Example:** - If data is sent at rate = 1Kbps then a noise of 1/100 sec can affect 10 bits. $(1/100 * 1000)$ □ If same data is sent at rate = 1Mbps then a noise of 1/100 sec can affect 10,000 bits. $(1/100 * 10^6)$

Error Detection

- Error detection means to decide whether the received data is correct or not without having a copy of the original message. Error detection uses the concept of redundancy, which means adding extra bits for detecting errors at the destination.

Error Detection

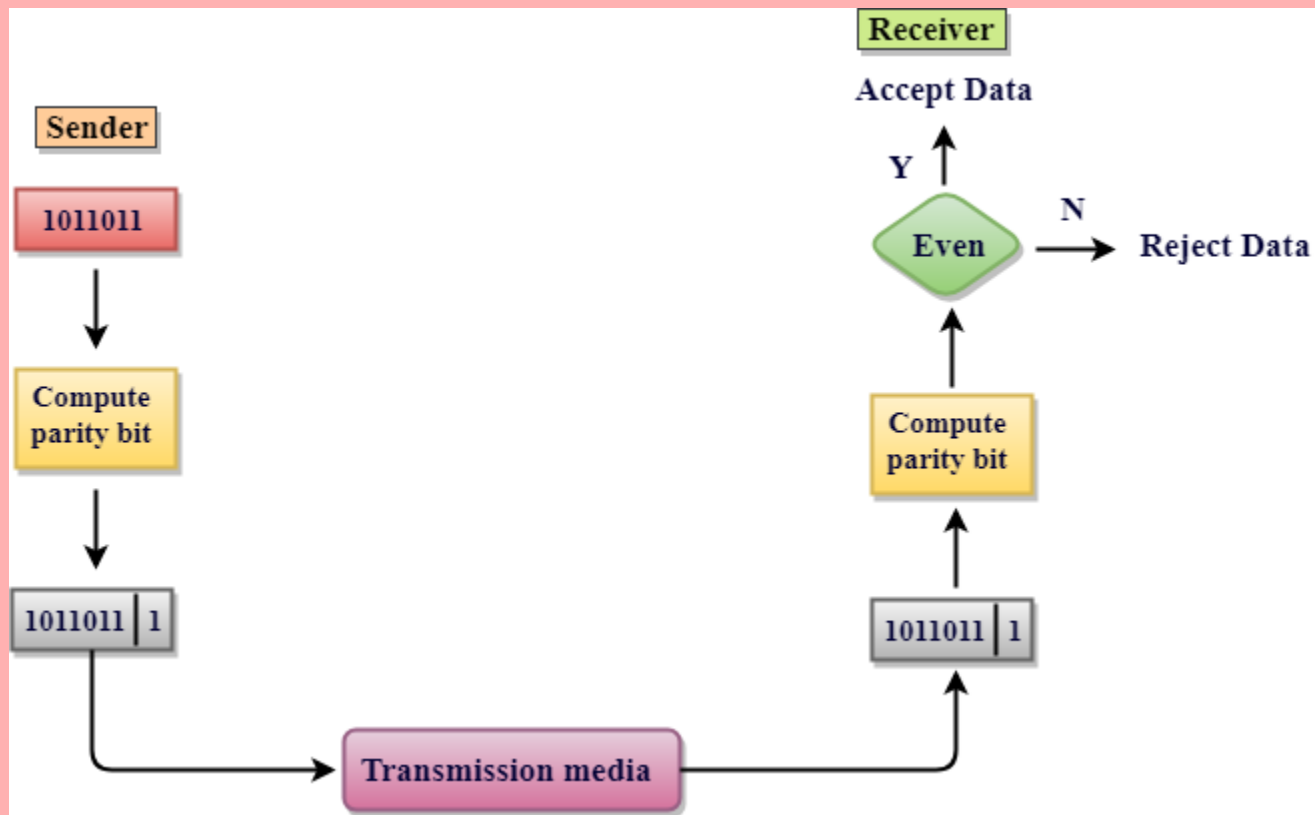
Types of Error Detection techniques

- Single parity check
- Two-dimensional parity check
- Checksum
- Cyclic redundancy check

Single parity check

- Single Parity checking is the simple mechanism and inexpensive to detect the errors.
- In this technique, a redundant bit is also known as a parity bit which is appended at the end of the data unit so that the number of 1s becomes even. Therefore, the total number of transmitted bits would be 9 bits.
- If the number of 1s bits is odd, then parity bit 1 is appended and if the number of 1s bits is even, then parity bit 0 is appended at the end of the data unit.
- At the receiving end, the parity bit is calculated from the received data bits and compared with the received parity bit.
- This technique generates the total number of 1s even, so it is known as even-parity checking.

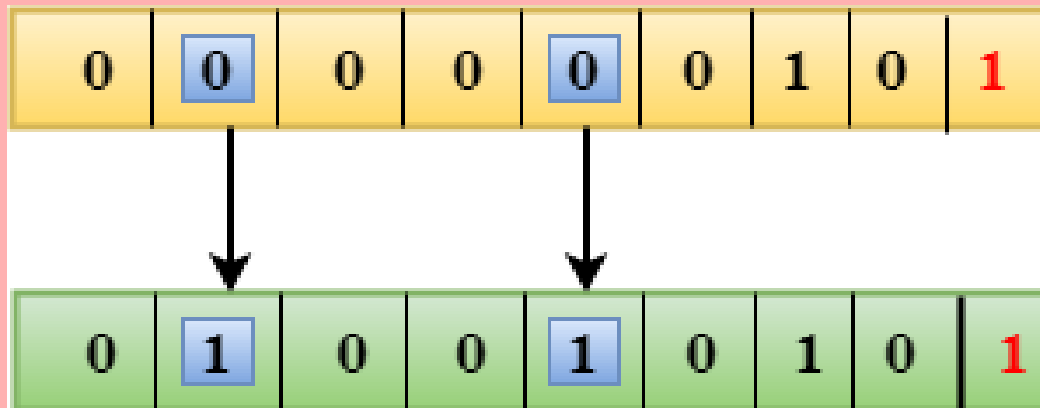
Single parity check



Single parity check

- Drawbacks Of Single Parity Checking
- It can only detect single-bit errors which are very rare.
- If two bits are interchanged, then it cannot detect the errors.

Single parity check



How Hamming Codes Work

- One parity bit can tell us an error occurred
- Multiple parity bits can also tell us where it occurred
- $O(\lg(n))$ bits needed to detect and correct one bit errors

Hamming (7, 4)

- 7 bits total
 - 4 data bits
 - 3 parity bits
- Can find and correct 1 bit errors
or
- Can find but not correct 2 bit errors

Hamming Bits Table

Bit position		1	2	3	4	5	6	7	...
Encoded data bits		p1	p2	d1	p3	d2	d3	d4	
Parity bit coverage	p1	X		X		X		X	
	p2		X	X			X	X	
	p3				X	X	X	X	

Example

Byte **1011 0001**

Two data blocks, **1011** and **0001**.

Expand the first block to 7 bits: **_ _ 1 _ 0 1 1**.

Bit 1 is 0, because $b_3 + b_5 + b_7$ is even.

Bit 2 is 1, $b_3 + b_6 + b_7$ is odd.

bit 4 is 0, because $b_5 + b_6 + b_7$ is even.

Our 7 bit block is: **0 1 1 0 0 1 1**

Repeat for right block giving **0 0 0 1**

Example

Byte **1011 0001**

Two data blocks, **1011** and **0001**.

Expand the first block to 7 bits: **__ 1 _ 0 1 1**.

Bit 1 is 0, because $b_3 + b_5 + b_7$ is even.

Bit 2 is 1, $b_3 + b_6 + b_7$ is odd.

bit 4 is 0, because $b_5 + b_6 + b_7$ is even.

Our 7 bit block is: **0 1 1 0 0 1 1**

Repeat for right block giving **1 1 0 1 0 0 1**

Example

Find Hamming bit for

i) **11011001**

ii) **00010100**

iii) **11111111**

iv) **01011010**

Hamming Code Syndrome Design Criteria

For convenience, we would like to generate a 4-bit syndrome for an 8-bit data word with the following characteristics:

- If the syndrome contains all 0s, no error has been detected.
- If the syndrome contains one and only one bit set to 1, then an error has occurred in one of the 4 check bits. No correction is needed.
- If the syndrome contains more than one bit set to 1, then the numerical value of the syndrome indicates the position of the data bit in error. This data bit is inverted for correction.

A Layout of Data and Check Bits that Achieves Our Design Criteria:

Bit position	12	11	10	9	8	7	6	5	4	3	2	1
Bit number	1100	1011	1010	1001	1000	0111	0110	0101	0100	0011	0010	0001
Data bit	D8	D7	D6	D5		D4	D3	D2		D1		
Check bit					C8				C4		C2	C1

C1 is a parity check on every data bit whose position is xxx1

$$C1 = D1 \text{ xor } D2 \text{ xor } D4 \text{ xor } D5 \text{ xor } D7$$

C2 is a parity check on every data bit whose position is xx1x

$$C2 = D1 \text{ xor } D3 \text{ xor } D4 \text{ xor } D6 \text{ xor } D7$$

C4 is a parity check on every data bit whose position is x1xx

$$C4 = D2 \text{ xor } D3 \text{ xor } D4 \text{ xor } D8$$

C8 is a parity check on every data bit whose position is 1xxx

$$C8 = D5 \text{ xor } D6 \text{ xor } D7 \text{ xor } D8$$

Why this ordering? Because we want the *syndrome*, the Hamming test word, to yield the address of the error.

Example:

Data stored = 00111001

Check bits:

C1 =

C2 =

C4 =

C8 =

Putting it together:

Example:

Data stored = 00111001

Check bits:

C1 =

C2 =

C4 =

C8 =

Putting it together:

Bit position	12	11	10	9	8	7	6	5	4	3	2	1
Position number	1100	1011	1010	1001	1000	0111	0110	0101	0100	0011	0010	0001
Data bit	D8	D7	D6	D5		D4	D3	D2		D1		
Check bit					C8				C4		C2	C1
Word stored as	0	0	1	1		1	0	0		1		

Example:

Data stored = 00111001

Check bits:

C1 =

C2 =

C4 =

C8 =

Putting it together:

Bit position	12	11	10	9	8	7	6	5	4	3	2	1
Position number	1100	1011	1010	1001	1000	0111	0110	0101	0100	0011	0010	0001
Data bit	D8	D7	D6	D5		D4	D3	D2		D1		
Check bit					C8				C4		C2	C1
Word stored as	0	0	1	1	0	1	0	0	1	1	1	1

Example:

Data stored = 00111001

Check bits:

$$C1 = 1 \oplus 0 \oplus 1 \oplus 1 \oplus 0 = 1$$

$$C2 = 1 \oplus 0 \oplus 1 \oplus 1 \oplus 0 = 1$$

$$C4 = 0 \oplus 0 \oplus 1 \oplus 0 = 1$$

$$C8 = 1 \oplus 1 \oplus 0 \oplus 0 = 0$$

Putting it together:

Bit position	12	11	10	9	8	7	6	5	4	3	2	1
Position number	1100	1011	1010	1001	1000	0111	0110	0101	0100	0011	0010	0001
Data bit	D8	D7	D6	D5		D4	D3	D2		D1		
Check bit					C8				C4		C2	C1
Word stored as	0	0	1	1	0	1	0	0	1	1	1	1

Example:

Let us verify that this scheme works with an example. Assume that the 8-bit input word is 00111001, with data bit D1 in the rightmost position. The calculations are as follows:

$$C1 = 1 \oplus 0 \oplus 1 \oplus 1 \oplus 0 = 1$$

$$C2 = 1 \oplus 0 \oplus 1 \oplus 1 \oplus 0 = 1$$

$$C4 = 0 \oplus 0 \oplus 1 \oplus 0 = 1$$

$$C8 = 1 \oplus 1 \oplus 0 \oplus 0 = 0$$

Suppose now that data bit 3 sustains an error and is changed from 0 to 1. When the check bits are recalculated, we have

$$C1 = 1 \oplus 0 \oplus 1 \oplus 1 \oplus 0 = 1$$

$$C2 = 1 \oplus 1 \oplus 1 \oplus 1 \oplus 0 = 0$$

$$C4 = 0 \oplus 1 \oplus 1 \oplus 0 = 0$$

$$C8 = 1 \oplus 1 \oplus 0 \oplus 0 = 0$$

When the new check bits are compared with the old check bits, the syndrome word is formed:

	C8	C4	C2	C1
	0	1	1	1
$\oplus$	0	0	0	1
	0	1	1	0

The result is 0110, indicating that bit position 6, which contains data bit 3, is in error.

Example:

Word fetched:

Bit position	12	11	10	9	8	7	6	5	4	3	2	1
Position number	1100	1011	1010	1001	1000	0111	0110	0101	0100	0011	0010	0001
Data bit	D8	D7	D6	D5		D4	D3	D2		D1		
Check bit					C8				C4		C2	C1
Word fetched as	0	0	1	1	0	1	1	0	1	1	1	1

Check Bits:

$$C1 = 1 \oplus 0 \oplus 1 \oplus 1 \oplus 0 = 1$$

$$C2 = 1 \oplus 1 \oplus 1 \oplus 1 \oplus 0 = 0$$

$$C4 = 0 \oplus 1 \oplus 1 \oplus 0 = 0$$

$$C8 = 1 \oplus 1 \oplus 0 \oplus 0 = 0$$

Comparing:

C8 C4 C2 C1

0 1 1 1 Orig Check Bits

0 0 0 1 New Check Bits

0 1 1 0 Syndrome

0110 = 6 → bit position 6 is wrong, i.e. bit D3 is wrong

Putting it all together:

Bit position	12	11	10	9	8	7	6	5	4	3	2	1
Position number	1100	1011	1010	1001	1000	0111	0110	0101	0100	0011	0010	0001
Data bit	D8	D7	D6	D5		D4	D3	D2		D1		
Check bit					C8				C4		C2	C1
Word stored as	0	0	1	1	0	1	0	0	1	1	1	1
Word fetched as	0	0	1	1	0	1	1	0	1	1	1	1
Position number	1100	1011	1010	1001	1000	0111	0110	0101	0100	0011	0010	0001
Check bit					0				0		0	1

Increased word length for error correcting

Data Bits	Single-Error Correction		Single-Error Correction/ Double-Error Detection	
	Check Bits	% Increase	Check Bits	% Increase
8	4	50	5	62.5
16	5	31.25	6	37.5
32	6	18.75	7	21.875
64	7	10.94	8	12.5
128	8	6.25	9	7.03
256	9	3.52	10	3.91

Hamming Code: Example

7-bit

Char.	ASCII	Check bits
H	1001000	 00110010000
a	1100001	
m	1101101	
m	1101101	
i	1101001	
n	1101110	
g	1100111	
	0100000	
c	1100011	
o	1101111	
d	1100100	
e	1100101	

Order of bit transmission

- . Hamming codes can only correct single errors.
- . But, to correct bursts of errors, send column by column.

Hamming Code: Example

7-bit

Char.	ASCII	Check bits
H	1001000	00110010000
a	1100001	10111001001
m	1101101	11101010101
m	1101101	11101010101
i	1101001	01101011001
n	1101110	01101010110
g	1100111	01111001111
	0100000	10011000000
c	1100011	11111000011
o	1101111	10101011111
d	1100100	11111001100
e	1100101	00111000101

Order of bit transmission

- . Hamming codes can only correct single errors.
- . But, to correct bursts of errors, send column by column.

Cyclic Redundancy Check (CRC)

- Mostly used in data communication
- Tells us whether an error has occurred, but does not correct the error.
- This is a type of “systematic error detection”
 - The error-checking bits are appended to the information byte

Cyclic Redundancy Check (CRC)

- Bit strings represented as polynomials with coef. 0 and 1.
- K bit frame $\Rightarrow x^{k-1} + \dots + 1$ (first and last coef must be 1)
- Example
 - 110001 $\Rightarrow x^5 + x^4 + 1$
- Polynomial arithmetic is done module 2 i.e.
 $\Leftrightarrow$ addition/subtraction = XOR operation

Cyclic Redundancy Check (CRC)

- Bit strings represented as polynomials with coef. 0 and 1.
- K bit frame $\Rightarrow x^{k-1} + \dots + 1$ (first and last coef must be 1)
- Example
 - 110001 $\Rightarrow x^5 + x^4 + 1$
- Polynomial arithmetic is done module 2 i.e.
 $\Leftrightarrow$ addition/subtraction = XOR operation

CRC

- Sender (S) and Receiver (R) agree on a *generator polynomial* $G(x)$
- Frame – m bits $\Rightarrow M(x)$ – the checksum of m is the remaining of $R(x)=M(x)/G(x)$
- Checksum added to frame.
- (R) Gets the frame $M'(x)=[M(x)-R(x)]$
 - If $M'(x)/G(x)$ has remainder $\Rightarrow$ **error**

Modulo 2 Arithmetic

- Addition Rules

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 0$$

Ex: 1011

$$\begin{array}{r} 1011 \\ +110 \\ \hline \end{array}$$

$$1101$$

Modulo 2 Arithmetic

- Division

Ex: $\overline{1011} \mid 1001011$

$\wedge$ $\wedge$

 divisor dividend

Modulo 2 Arithmetic

- Division

Ex:
$$\begin{array}{r} 1 \\ \hline 1011 \mid 1001011 \\ \underline{1011} \end{array}$$

Modulo 2 Arithmetic

- Division

Ex:
$$\begin{array}{r} \overline{1} \\ 1011 \mid 1001011 \\ \underline{1011} \\ 0010 \end{array}$$

Modulo 2 Arithmetic

- Division

Ex:
$$\begin{array}{r} \overline{10} \\ 1011 \mid 1001011 \\ \underline{1011} \\ 00100 \end{array}$$

Modulo 2 Arithmetic

- Division

Ex: **1011** | **1001011**

$$\begin{array}{r} \overline{101} \\ 1011 \overline{) 1001011} \\ \underline{1011} \\ 001001 \end{array}$$

Modulo 2 Arithmetic

- Division

Ex: **1011** | **1001011**

$$\begin{array}{r} \underline{101} \\ 1011 \\ \underline{1011} \\ 001001 \\ \underline{1011} \end{array}$$

Modulo 2 Arithmetic

- Division

Ex: **1011** | **1001011**

$$\begin{array}{r} \underline{101} \\ 1011 \\ \underline{1011} \\ 001001 \\ \underline{1011} \\ 0010 \end{array}$$

Modulo 2 Arithmetic

- Division

Ex: **1011** | **1001011**

$$\begin{array}{r} \underline{1010} \\ 1011 \overline{) 1001011} \\ \underline{1011} \\ 001001 \\ \underline{1011} \\ 00101 \end{array}$$

Modulo 2 Arithmetic

- Division

Ex: **1011** | **1001011**

$$\begin{array}{r} \underline{1010} \\ 1011 \overline{) 1001011} \\ \underline{001001} \\ \underline{1011} \\ 00101 \end{array}$$

 ☐ Quotient

 ☐ Remainder

Calculating and Using CRCs

1. Let the information byte $F = 1001011$
2. The sender and receiver agree on an arbitrary binary pattern P . Let $P = 1011$.
3. Shift F to the left by 1 less than the number of bits in P .
Now, $F = 1001011000$.
4. Let F be the dividend and P be the divisor. Perform “modulo 2 division”.
5. After performing the division, we ignore the quotient. We got 100 for the remainder, which becomes the actual CRC checksum.
6. Add the remainder to F , giving the message M :
 $1001011 + 100 = 1001011100 = M$

Cyclic Redundancy Code(CRC)

- CRC or Cyclic Redundancy Check is a method of detecting accidental changes/errors in the communication channel.
CRC uses **Generator Polynomial** which is available on both sender and receiver side

Cyclic Redundancy Code(CRC)

- **Sender Side (Generation of Encoded Data from Data and Generator Polynomial (or Key)):**
 1. The binary data is first augmented by adding $k-1$ zeros in the end of the data
 2. Use ***modulo-2 binary division*** to divide binary data by the key and store remainder of division.
 3. Append the remainder at the end of the data to form the encoded data and send the same
- **Receiver Side (Check if there are errors introduced in transmission)**

Perform modulo-2 division again and if the remainder is 0, then there are no errors.

Calculating and Using CRCs

7. M is decoded and checked by the message receiver using the reverse process.

$$\begin{array}{r} \underline{ 1010100} \\ 1011 \mid 1001011100 \\ \underline{ 1011} \\ 001001 \\ 1001 \\ \underline{0010} \\ 001011 \\ \underline{1011} \\ 0000 \end{array}$$

☐ Remainder

CRC – Example, Transmitted Frame

Frame : 1 1 0 1 0 1 1 0 1 1

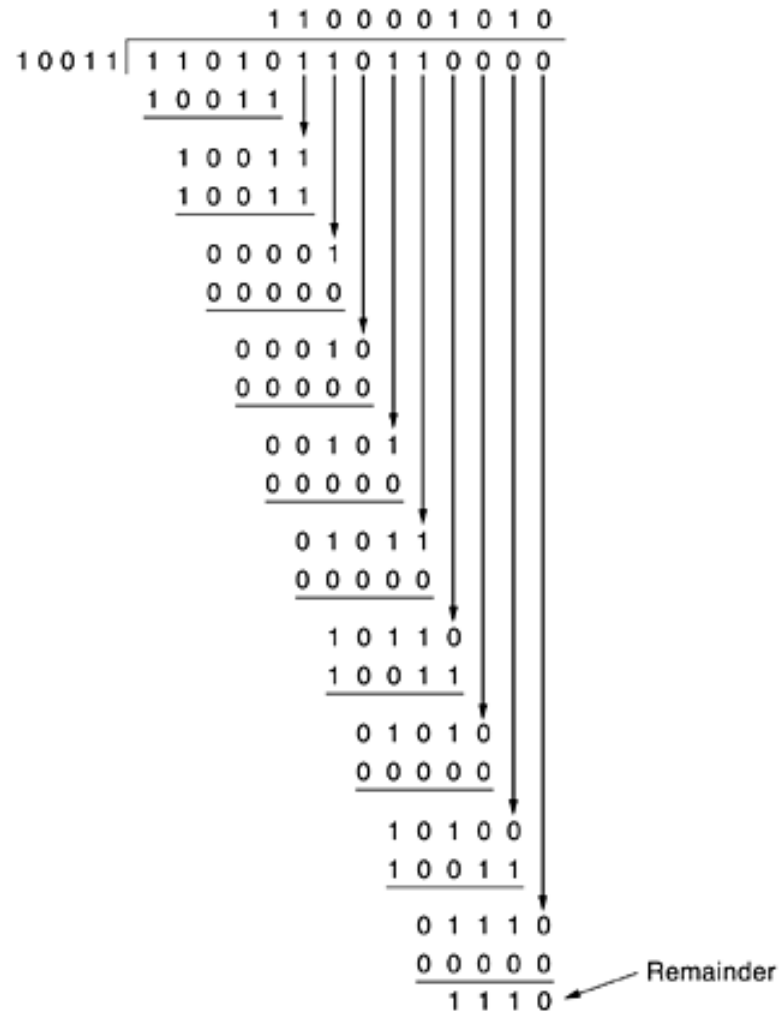
Generator: 1 0 0 1 1

CRC - Example

Frame : 1 1 0 1 0 1 1 0 1 1

Generator: 1 0 0 1 1

Message after 4 zero bits are appended: 1 1 0 1 0 1 1 0 1 1 0 0 0 0



Frame – 1101011011

$$G(x) = x^4 + x + 1$$

Transmitted frame:

11010110110000 –

00000000001110

11010110111110

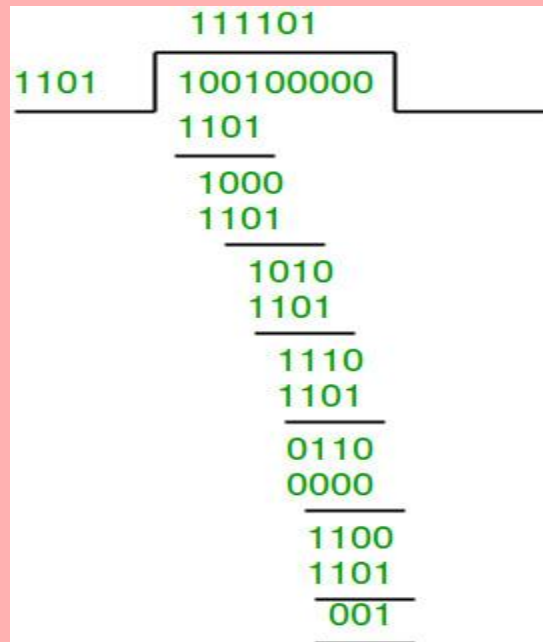
Transmitted frame: 1 1 0 1 0 1 1 0 1 1 1 1 1 0

CRC Example

Data word to be sent - 100100

Key - 1101

Sender Side:



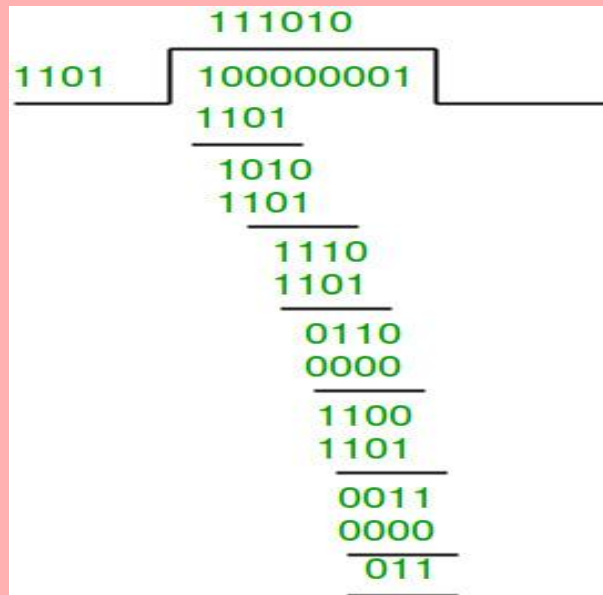
Therefore, the remainder is 001 and hence the code word sent is 100100001.

CRC Example

Receiver Side

Let there be an error in transmission media

Code word received at the receiver side - 100000001



Since the remainder is not all zeroes, the error is detected at the receiver side.

Checksum Error Detecting Method

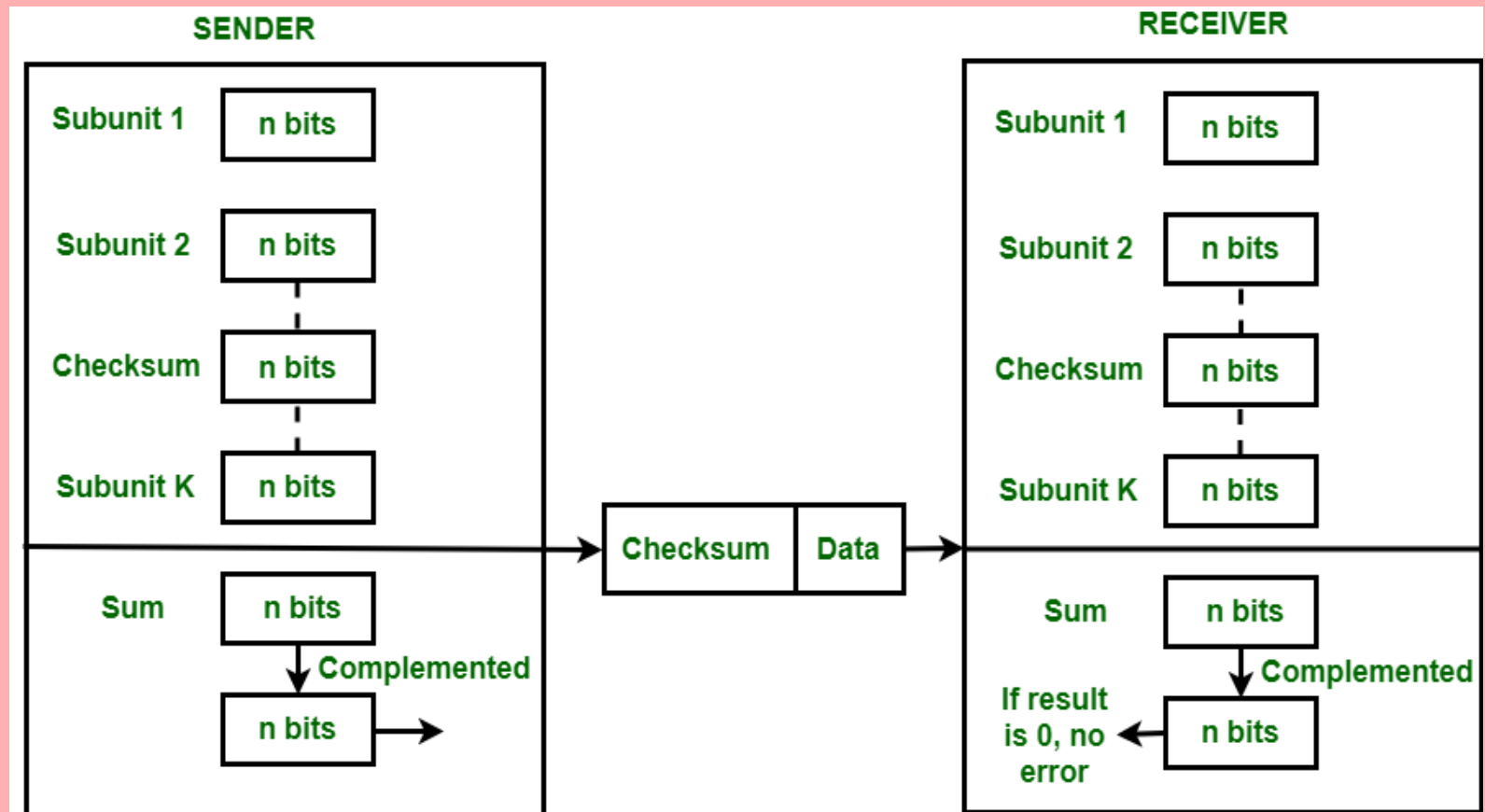
The Checksum is an **error detection** technique that detects an error in a transmitted message by appending redundant bits in the message. This method makes the use of Checksum Generator on Sender side and **Checksum Checker** on Receiver side.

Checksum Error Detecting Method

- Checksum is used for error detection.
- Checksum is the redundant bits that are attached with actual data.
- Sometimes, a Checksum is also considered as a hash sum or hash value.
- On the sender's side, this method uses a checksum generator to generate a Checksum. On the receiving end, a checksum checker is used to validate whether the correct data is received.
- Checksum is an error detection algorithm that can be applied to any length message.
- Checksum provides information to the receiver about the transmission to ensure that the full range of data is delivered successfully.

Checksum Error Detecting Method

At the Sender side, the data is divided into equal subunits of n bit length by the checksum generator. This bit is generally of 16-bit length. These subunits are then added together using one's complement method. This sum is of n bits. The resultant bit is then complemented. This complemented sum which is called checksum is appended to the end of original data unit and is then transmitted to receiver.



Checksum Error Detecting Method

- The Receiver after receiving data + checksum passes it to checksum checker. Checksum checker divides this data unit into various subunits of equal length and adds all these subunits.
- These subunits also contain checksum as one of the subunits. The resultant bit is then complemented.
- If the complemented result is zero, it means the data is error-free. If the result is non-zero it means the data contains an error and Receiver rejects it.

Types of Checksum Algorithms

- **SHA-0 :**

SHA-0 was the first hash function of this type, but it was withdrawn briefly in 1993.

- **SHA-1 :**

SHA-1 is a hash function that is not considered a secure hash function as of 2010.

- **SHA-2 (224, 256, 384, 512) :**

This includes some changes from its previous version, SHA-1. The SHA-2 family consists of a total of six hash functions with hash values that are 224, 256, 384, or 512 bits.

- **MD5 :**

MD5 is a hash function that generates a checksum value, but it is not required for each file to have a unique number. So MD5 is open to vulnerabilities if the hacker swaps the file with a checksum value.

Steps to perform the Checksum

Checksum at Sender Side

The sender divides data into blocks of equal size and then adds the data of every block using 1's complement arithmetic to get the sum. It then complements the sum to get the Checksum and sends it along with the data frames. At sender side steps of generation of the Checksum are given below :

- **Step 1 :**

First of all, break the given data into “k” an equal number of blocks, i.e., “N” bits in all the “K” blocks of the message.

- **Step 2 :**

Perform addition of all the “k” divisions

- **Step 3 :**

If there is a carry bit, add it.

- **Step 4 :**

Now find the 1's complement of the sum. For finding 1's complement of any binary number, just replace every zero with one and replace everyone with zero(e.g., 10001 - 1's Complement 01110) This complemented sum is known as **Checksum**.

- **Step 5 :**

Checksum is appended to the message to be sent to the receiver.

Steps to perform the Checksum

Receiver Side

Step 1 :

Perform the addition on all the k's-size blocks.

Step 2 :

If any carry is generated in performing an addition, then sum it with the step 1 result.

Below is an image to show the addition of carrying to the sum.

Step 3 :

Find the 1's complement of a sum. If all bits are zero in the result, then simply **ACCEPT** that message, otherwise, **REJECTS** the data.

Below is an image to show the 1's complement of the sum.

Example of Checksum 1

Suppose that the sender wants to send 4 frames each of 8 bits, where the frames are 11001100, 10101010, 11110000 and 11000011.

What is the value of checksum ???

Example of Checksum

Suppose that a message 1001 1100 1010 0011 is transmitted using Internet Checksum (4-bit word). What is the value of the checksum?

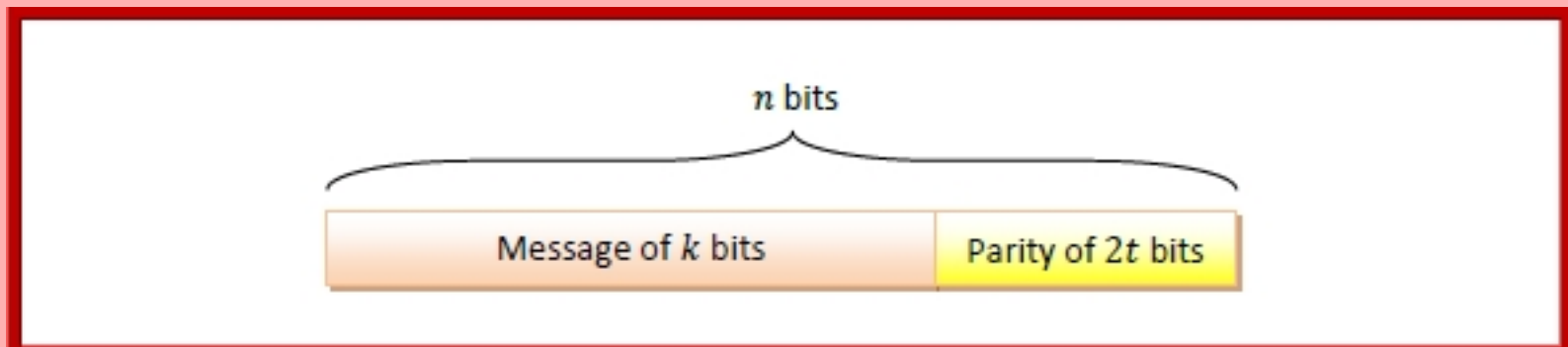
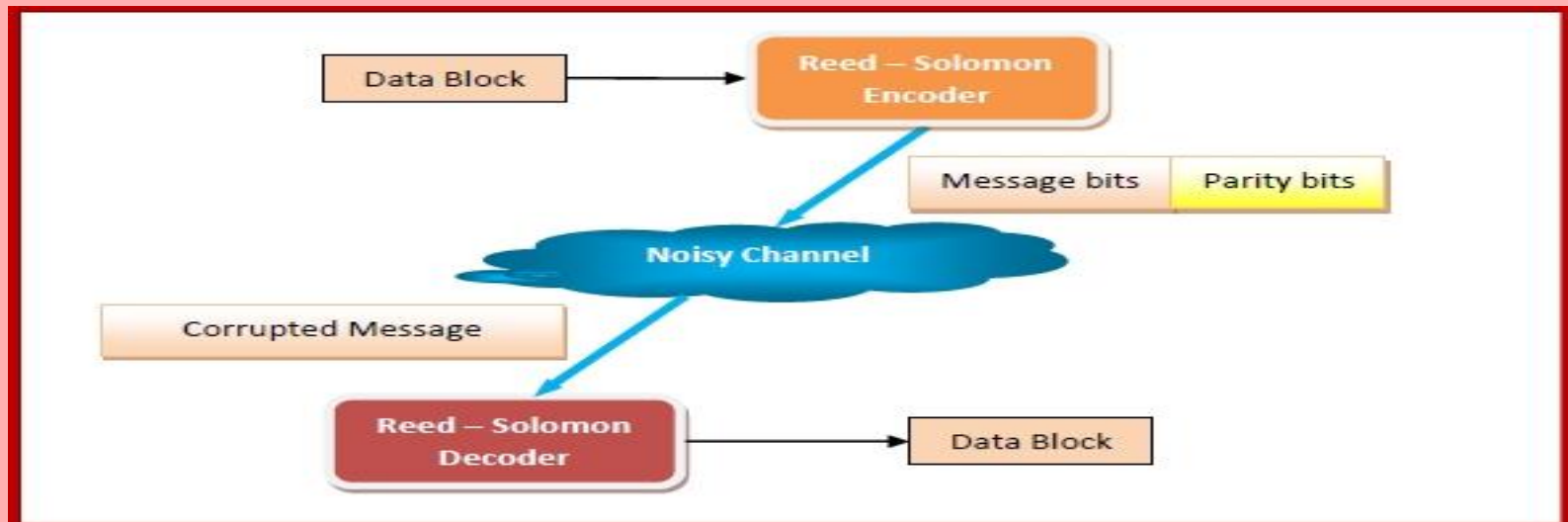
Error Correction

1. **Hamming codes.**
2. **Reed-Solomon codes.**

Reed-Solomon (RS)

It is a subclass of non - binary BCH codes. BCH codes (Bose-Chaudhuri-Hocquenghem codes) are cyclic ECCs that are constructed using polynomials over data blocks.

A Reed - Solomon encoder accepts a block of data and adds redundant bits (parity bits) before transmitting it over noisy channels. On receiving the data, a decoder corrects the error depending upon the code characteristics.



Reed-Solomon (RS)

Encoding using Reed Solomon Code

- The message is represented as a polynomial $p(x)$, and then multiplied with the generator polynomial $g(x)$.
- The message vector $[x_1, x_2, x_3, \dots, x_k]$ is mapped to a polynomial of degree less than k such that $p_x(\alpha_i) = x_i$ for all $i = 1, \dots, k$
- The polynomial is evaluated using interpolation methods like Lagrange Interpolation.
- Using this polynomial, the other points $\alpha_{k+1}, \dots, \alpha_n$, are evaluated.
- The encoded message is calculated as $s(x) = p(x) * g(x)$. The sender sends this encoded message along with the generator polynomial $g(x)$.

Reed-Solomon (RS)

Decoding using Reed Solomon Code

- The receiver receives the message $r(x)$ and divides it by the generator polynomial $g(x)$.
- If $r(x)/g(x)=0$, then it implies no error.
- If $r(x)/g(x)\neq 0$, then the error polynomial is evaluated using the expression: $r(x) = p(x) * g(x) + e(x)$
- The error polynomial gives the error positions.

Reed-Solomon (RS)

Properties of Reed-Solomon code:-

1. Block Length , $n = 2^m - 1$ symbols.
2. Message Size K symbols.
3. Parity Check Size $(n - k) = 2^t$ symbols.
4. Minimum distance , $d_{\min} = (2t + 1)$.

(n, k) code is used to encode m -bit symbols.

Reed-Solomon (RS)

- It is a subclass of Non-Binary BCH Codes.

The encoder for an RS code differs from a binary encoder in that it operates on the multiple bits rather than individual bits.

Properties of Reed-Solomon code:-

1. Block Length , $n = 2^m - 1$ symbols.
2. Message Size K symbols.
3. Parity Check Size $(n - k) = 2^t$ symbols.
4. Minimum distance , $d_{\min} = (2t + 1)$.
5. Parity Bit Size = $n - k = 2t$

(n, k) code is used to encode m -bit symbols.

Redundant bits = $(n - k)$

Reed-Solomon (RS)

Application Areas of Reed Solomon Codes:-

1. Storage devices [CD, DVD]
2. Wireless (Mobile Communication)
3. Satellite Communication
4. Digital TV
5. High Speed Modems.
6. Bar Code

Elementary Data Link Protocol

Summary

- 3 Types of Error Detection/Correction Methods
 - **Cyclic Redundancy Check (CRC)**
 - Used primarily in communication
 - Can only detect errors.
 - **Hamming Codes**
 - Can detect and correct errors.
 - The more parity bits added, the more errors can be detected and corrected.
 - Used to detect errors in memory bits or fixed magnetic disk drives, in which errors occur randomly.

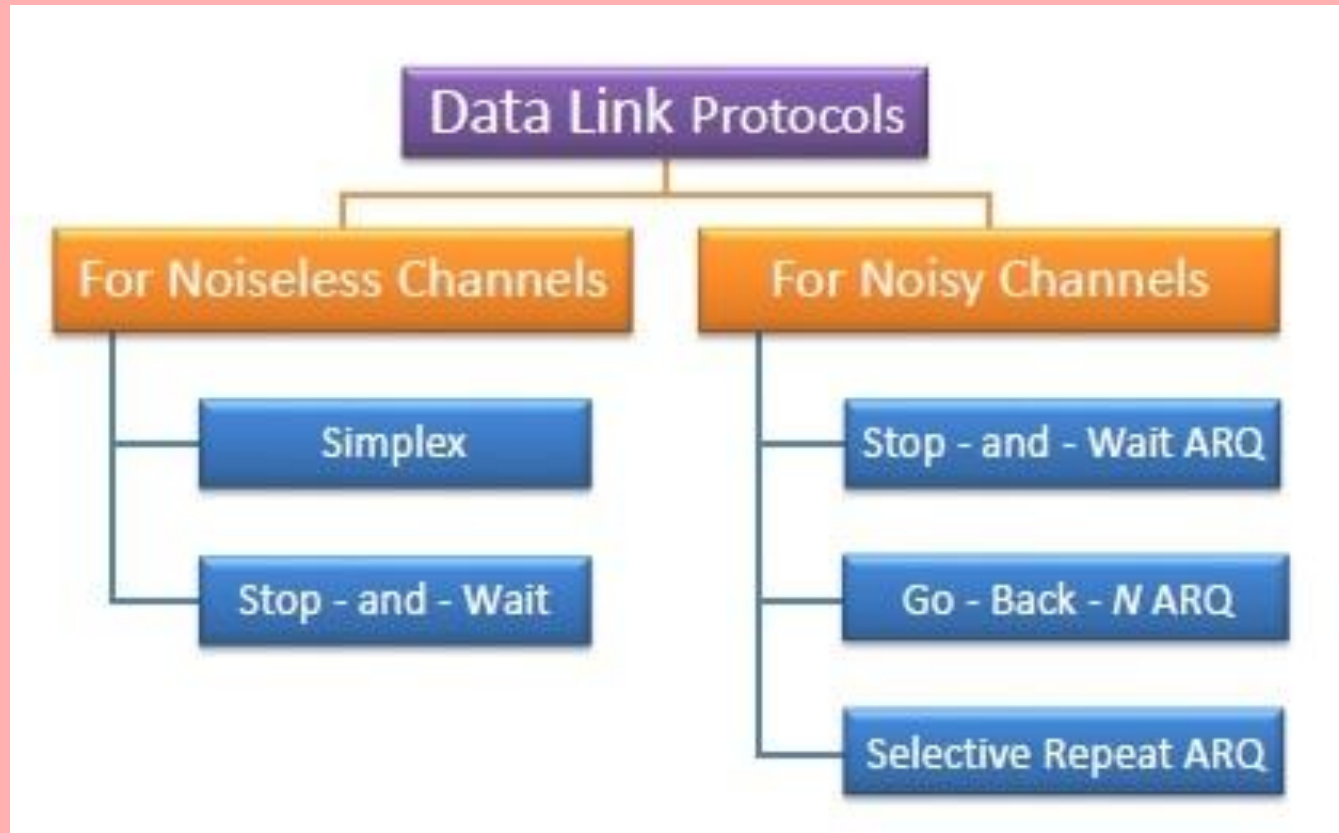
Summary

- **Reed-Solomon (RS)**
 - Can detect errors that occur in blocks (adjacent errors)
 - Used to detect errors in removable media such as magnetic tapes or compact disks, in which “burst errors” occur due to mishandling and environmental stress.

Elementary Data Link Protocols

1. Protocols in the data link layer are designed so that this layer can perform its basic functions: **framing, error control and flow control**.
2. Framing is the process of dividing bit - streams from physical layer into data frames whose size ranges from a few hundred to a few thousand bytes.
3. Error control mechanisms deals with transmission errors and retransmission of corrupted and lost frames.
4. Flow control regulates speed of delivery and so that a fast sender does not drown a slow receiver.

Elementary Data Link Protocols



Simplest Protocols

- A simple protocol for a noiseless channel would be one that involves the direct transfer of data from the source to the destination without any intermediate processing.
- In this scenario, the channel is assumed to be noise-free, which means that the data transmitted remains intact and does not get corrupted. In a noiseless channel, a simple protocol could consist of a straightforward method for transmitting data, such as sending one bit at a time, with no error correction or flow control mechanisms in place.

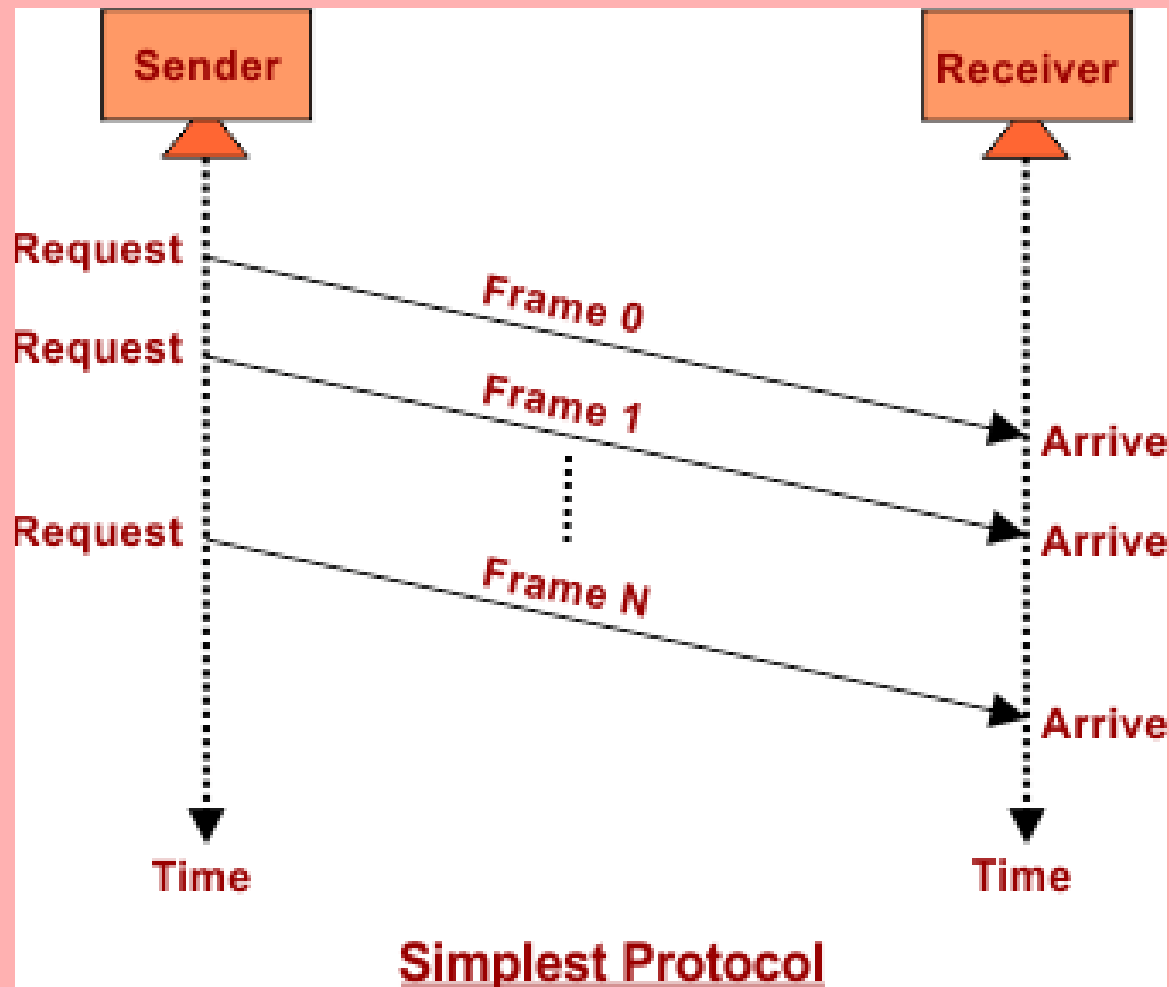
Simplest Protocols

- There is no noise whatsoever in the broadcast channel (a channel with no duplications, lost frames, or corrupted frames).
- The assumption is that the transmission route is perfect, with no instances of data loss, duplication, or corruption.
- There is no system for controlling errors and flow.
- The sender's and receiver's end have an endless amount of buffer space for the frames.

Simplest Protocols

- The processing time for this technique is quite brief. So, it may be disregarded.
- Each party is constantly prepared to send and receive data.
- Without giving the recipient any thought, the sender delivers a series of data frames.
- No data loss occurs, hence neither an ACK nor a NACK is sent.

Simplest Protocols

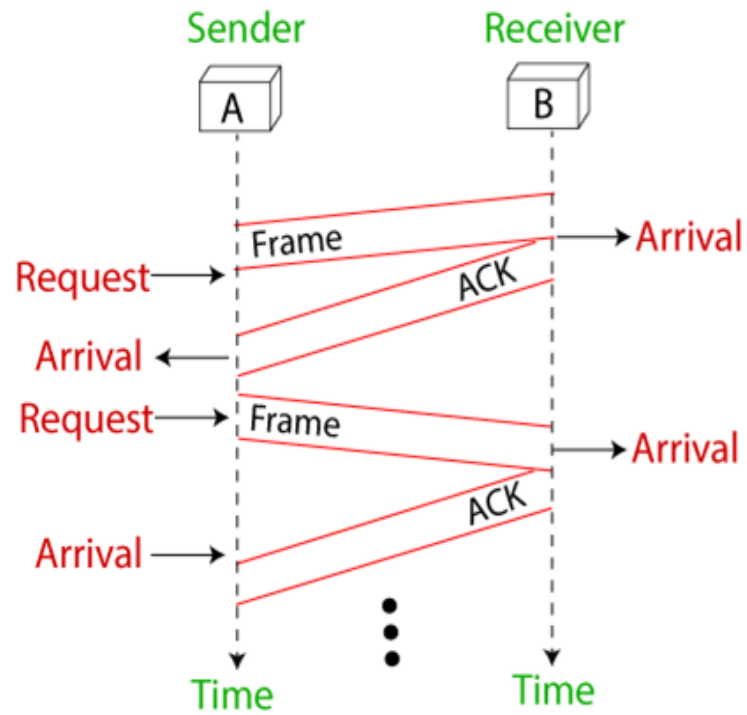


Stop and Wait Protocols

- Stop and wait is a protocol that is used for reliable data transmission in a noiseless channel. In this protocol, the sender sends a single packet at a time and waits for an acknowledgment (ACK) from the receiver before sending the next packet.
- This way, the sender can ensure that each packet is received by the receiver and has been successfully processed. If the sender does not receive an ACK within a certain time frame, the packet is considered lost and must be retransmitted.

Stop and Wait Protocols

- The flow of data frames at the receiver side may become too fast for it to be processed, causing the need for temporary storage.
- However, the limited storage space of the receiver may result in the loss or discarding of frames, or even denial of service.
- To prevent this, the sender must slow down their rate of transmission, which is achieved through the use of ACK messages from the receiver.
- The sender sends a single frame and waits for confirmation from the receiver before sending the next one, adding flow control to the previous protocol.
- The communication remains unidirectional for data frames, but ACK frames flow in the opposite direction.



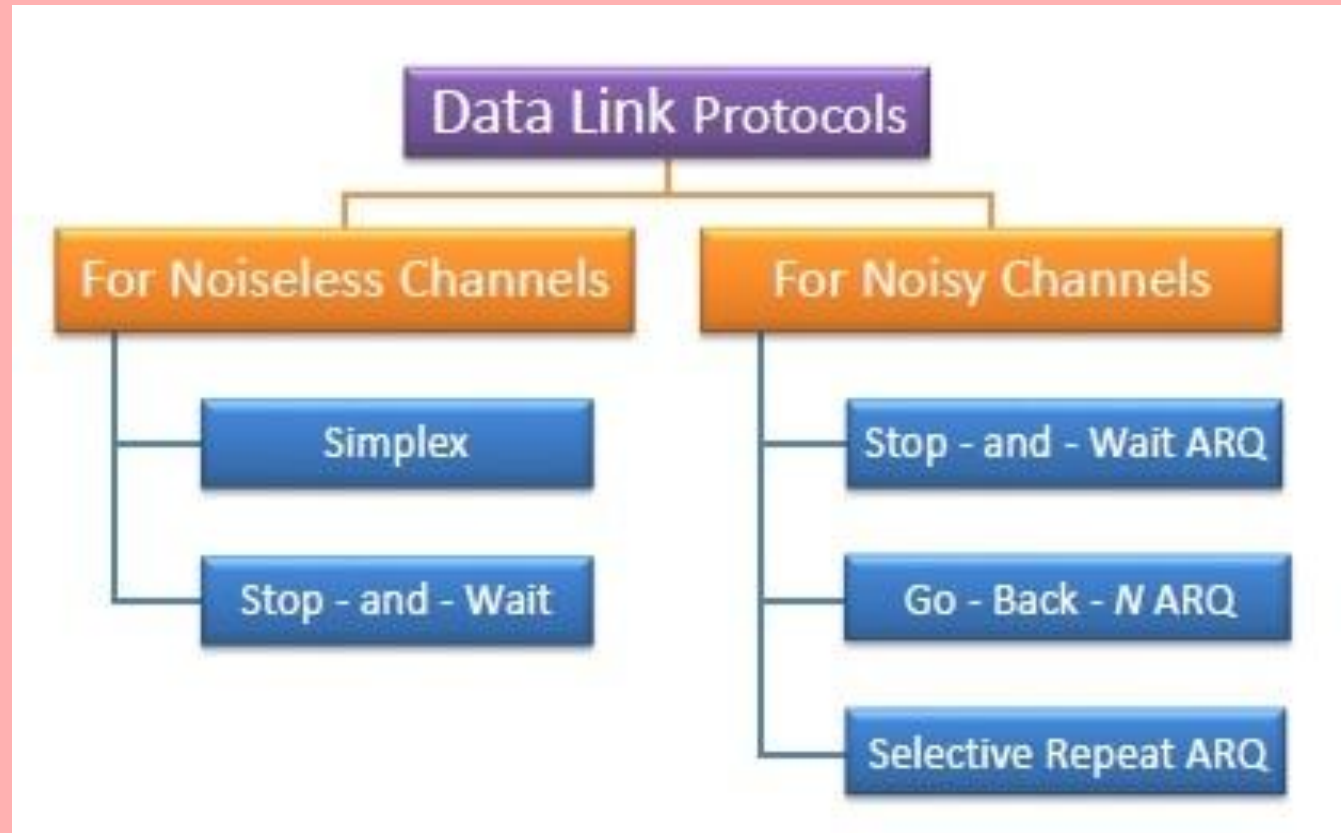
1. The sender transmits a data frame to the receiver.
2. The sender waits for an acknowledgment (ACK) from the receiver.
3. The receiver processes the received data frame.
4. The receiver sends an ACK to the sender to confirm receipt of the data frame.
5. The sender continues to transmit the next data frame, repeating the process from step i.

.

Difference

Simplest Protocol	Stop and Wait Protocols
1. Simplest Protocol has no flow control and error control mechanisms	1. Stop-and-Wait Protocol employs a flow control mechanism through the use of ACK frames.
2. The recipient is always expected to be ready to receive any frames sent by the sender, so no acknowledgment is needed	2. In the Stop-and-Wait Protocol, the sender must wait for an acknowledgment from the receiver before sending the next frame.
3. Simplest Protocol is unidirectional	Stop-and-Wait Protocol is bi-directional.

Noisy Channel



NOISY CHANNELS (Sliding Window Protocol)

The error control mechanism is used so that the received data should be exactly same whatever sender has sent the data. The error control mechanism is divided into two categories, i.e.,

1. Stop and Wait ARQ and sliding window.

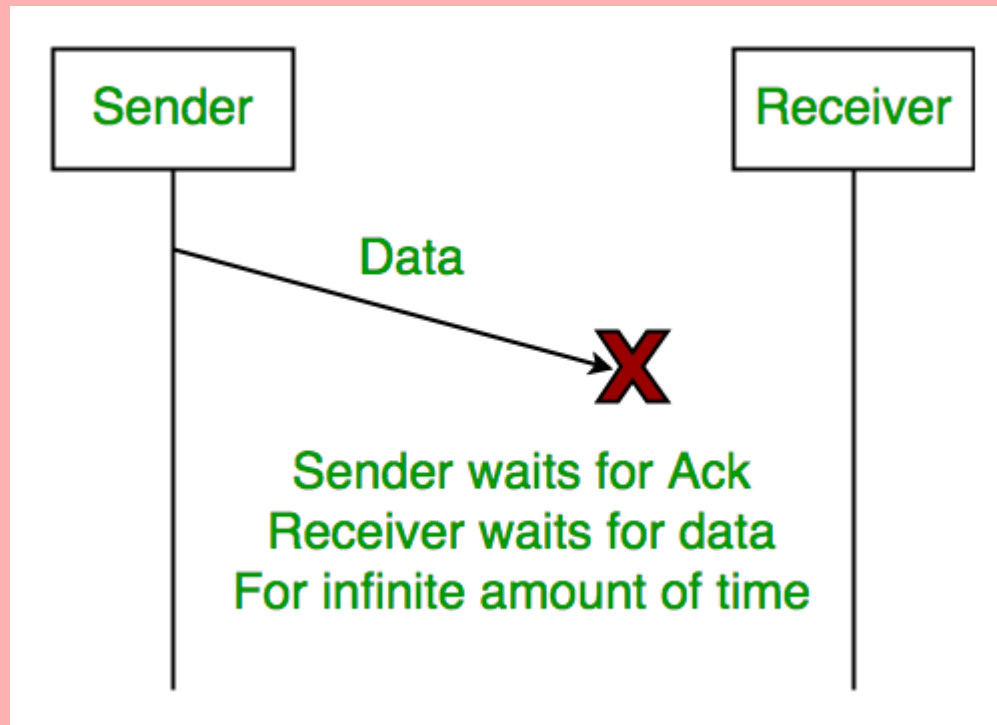
The sliding window is further divided into two categories, i.e.,

1. Go Back N, and
2. Selective Repeat. Based on the usage, the people select the error control mechanism whether it is stop and wait or sliding window.

Problems Associated with Stop and Wait

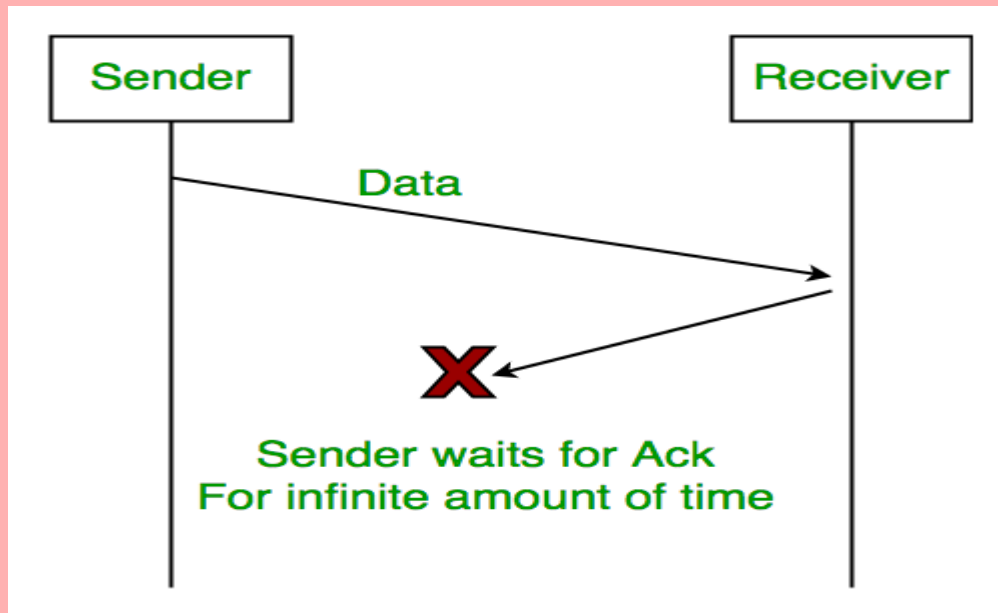
1. Lost Data

Assume the sender transmits the data packet and it is lost. The receiver has been waiting for the data for a long time. Because the data is not received by the receiver, it does not transmit an acknowledgment. The sender does not receive an acknowledgment, it will not send the next packet. This problem is caused by a loss of data.



2. Lost Acknowledgement

Assume the sender sends the data, which is also received by the receiver. The receiver sends an acknowledgment after receiving the packet. In this situation, the acknowledgment is lost in the network. The sender does not send the next data packet because it does not receive acknowledgement, under the stop and wait protocol, the next packet cannot be transmitted until the preceding packet's acknowledgment is received.



3. Delayed Acknowledgement/Data.

Assume the sender sends the data, which is also received by the receiver. The receiver then transmits the acknowledgment, which is received after the sender's timeout period. After a timeout on the sender side, a long-delayed acknowledgement might be wrongly considered as acknowledgement of some other recent packet.

Stop and Wait for ARQ (Automatic Repeat Request)

Stop (and) Wait + Time Out + Sequence No.(Data) + Sequence No. (ACK)

↑
Lost Data

↑
Lost Ack

↑
Delayed Ack

Stop and Wait for ARQ (Automatic Repeat Request)

The above 3 problems are resolved by Stop and Wait for ARQ (Automatic Repeat Request) that does both error control and flow control.

1. Time Out

Timeout refers to the duration for which the sender waits for an acknowledgment (ACK) from the receiver after transmitting a data packet. If the sender does not receive an ACK within this timeout period, it assumes that the frame was lost or corrupted and retransmits the frame.

2. Sequence Number (Data)

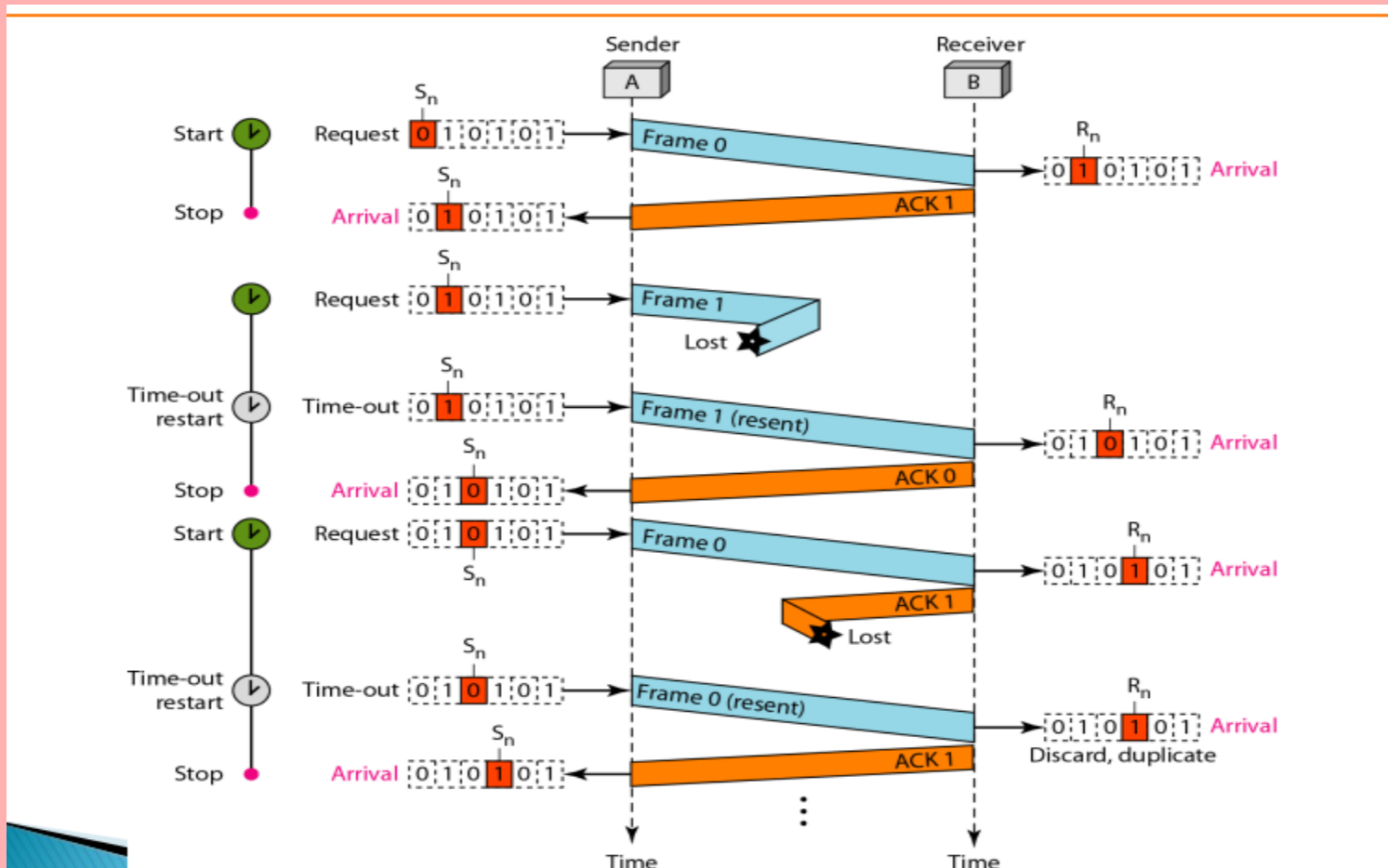
In Stop-and-Wait ARQ, the sender assigns sequence numbers to each data frame it sends. This allows the receiver to identify and acknowledge each frame individually, ensuring reliable delivery of data packets. After sending a frame, the sender waits for an acknowledgment before sending the next frame.

Stop and Wait for ARQ (Automatic Repeat Request)

3. Sequence Number(Acknowledgement)

Similarly, sequence numbers are also used in acknowledgments (ACKs) sent by the receiver to acknowledge received data frames. When the receiver successfully receives a data frame, it sends an ACK back to the sender, indicating the sequence number of the next expected frame. The sender uses this ACK to determine whether the transmission was successful and whether it can proceed to send the next frame.

Flow diagram for Stop and Wait ARQ



Networking Concept

In networking, **transmission delay** is a key factor that affects how data is transferred from one point to another. However, in a broader context, **delay** in a network encompasses several types, each impacting the overall time it takes for data to travel across a network. Here's a detailed explanation of the different types of delays:

1. Transmission Delay

Definition:

- Transmission delay (or transmission time) is the time required to push all the bits of a packet or frame into the transmission medium.

Formula:

Transmission Delay = Transmission Rate (in bits per second) / Size of Packet (in bits)

Networking Concept

2. Propagation Delay

Definition:

- Propagation delay is the time it takes for a signal to travel from the sender to the receiver through the transmission medium.

Formula:

Propagation Delay = $\frac{\text{Distance}}{\text{Propagation Speed}}$ \ \text{Propagation Delay}

Networking Concept

3. Queueing Delay

Definition:

- **Queueing delay** is the time a packet spends waiting in a buffer or queue at a router or switch before being transmitted.

Factors Affecting Queueing Delay:

- **Network Congestion:** More packets in the network lead to higher queueing delays.
- **Buffer Size:** Larger buffers can accommodate more packets but may introduce more delay if the buffer is full.

4. Processing Delay

Definition:

- **Processing delay** is the time required for a router or switch to process a packet header and determine where to forward the packet.

Summary of Delay Types

1. **Transmission Delay:** Time to push all bits of a packet into the transmission medium.
2. **Propagation Delay:** Time for a signal to travel from sender to receiver.
3. **Queueing Delay:** Time spent in queues at routers or switches.
4. **Processing Delay:** Time spent processing packet headers and other tasks at routers or switches.

Total Delay Calculation

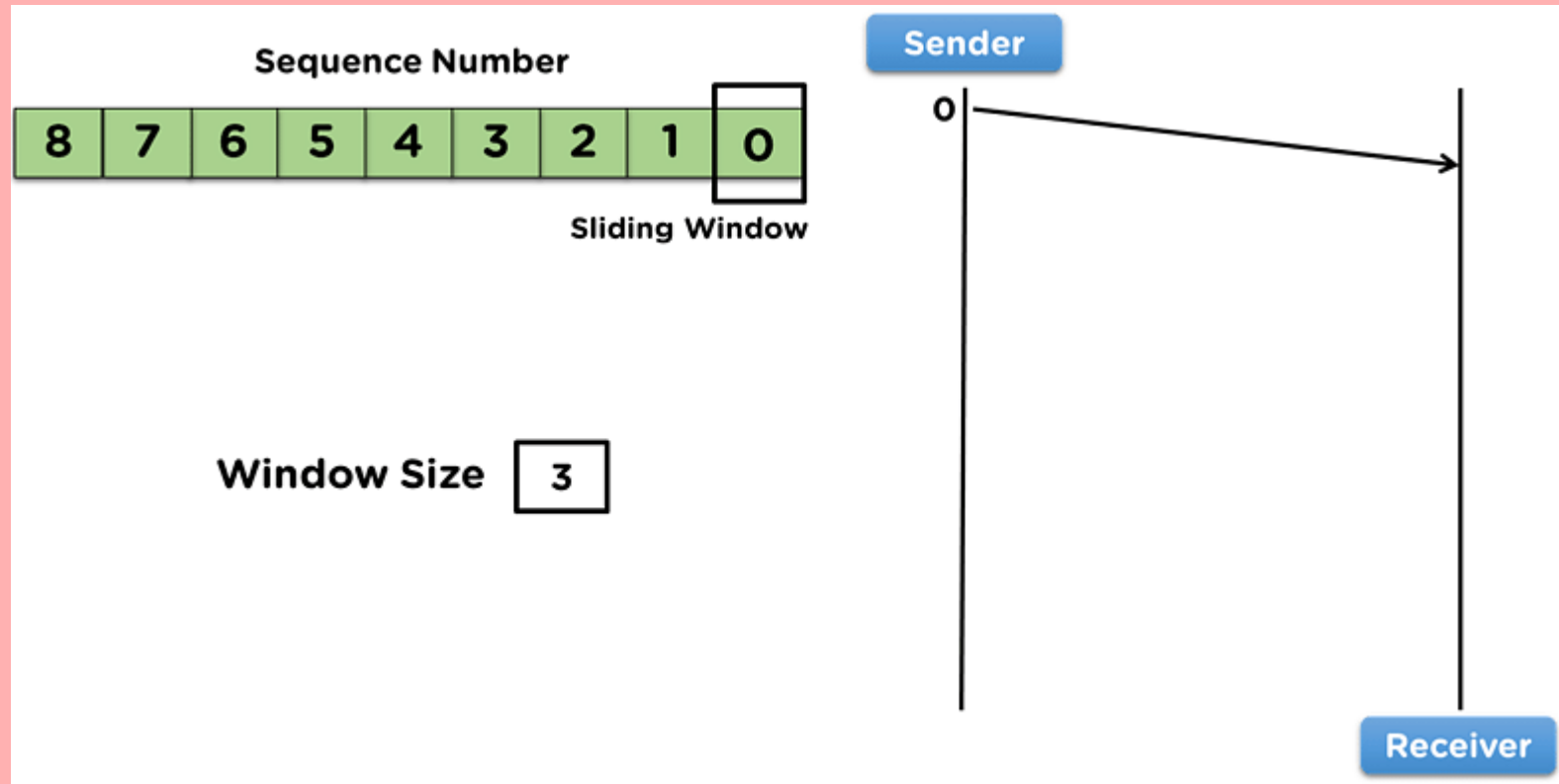
The total time for a packet to traverse a network segment is the sum of all these delays:

Total Delay=Transmission Delay+Propagation Delay+Queueing Delay+Processing Delay

Sliding Window Protocol

- The sliding window (windowing) technique is used by Transmission Control Protocol ([TCP](#)) to manage the flow of packets between two computers or network hosts. TCP is a core component of the [Internet Protocol suite](#) and operates at the transport layer.

What Is Sliding Window Protocol?



In the sliding window protocol, some of the most important features that are applied in a network model are as mentioned:

- This protocol allows sharing multiple data frames from the sender before receiving any acknowledgment from the receiver side.
- The data frames shared in the channel are defined by the window size mentioned in the network model, which defines the maximum number of frames that can be transmitted at a time from the sender to the receiver side before expecting any acknowledgment.
- Each of the frames in the network model is assigned a sequence number to increase the transmission efficiency.
- The data frames shared from the sender to the receiver side are enclosed within a virtual sliding window, which represents that these are awaiting acknowledgment from the receiver side.

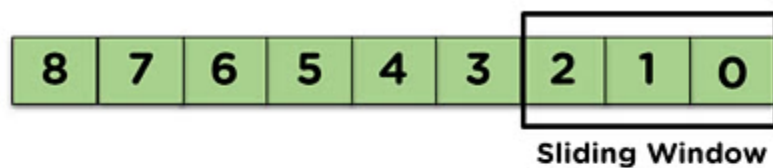
Working of the Sliding Window Protocol

- The working of the sliding window protocol can be divided into two steps sender steps, and the receiver steps and also some important values are needed in a network model for smooth transmission of the data frames are:
 - **Sender and the receiver side**
 - **Window Size**
 - **The total data frames to be transmitted**
 - **Proper sequencing of the frames**

Steps for the Sender Side

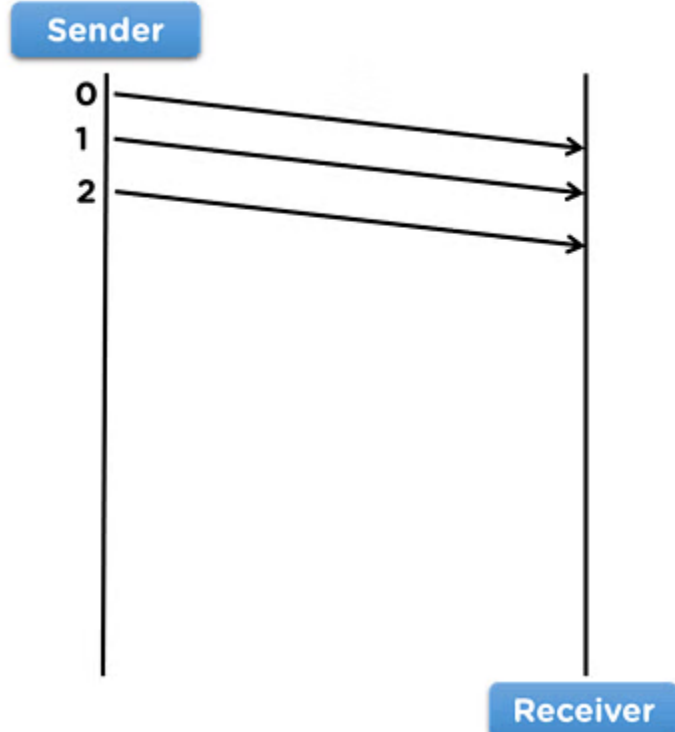
Step 1:-

- To begin with, the sender side will share data frames with the receiver side per the window size assigned to the model.
- The sliding window will appear on the frames transmitted over to the receiver side.
- Then the sender will wait for an acknowledgment from the receiver side for the shared frames, as mentioned in figure1.

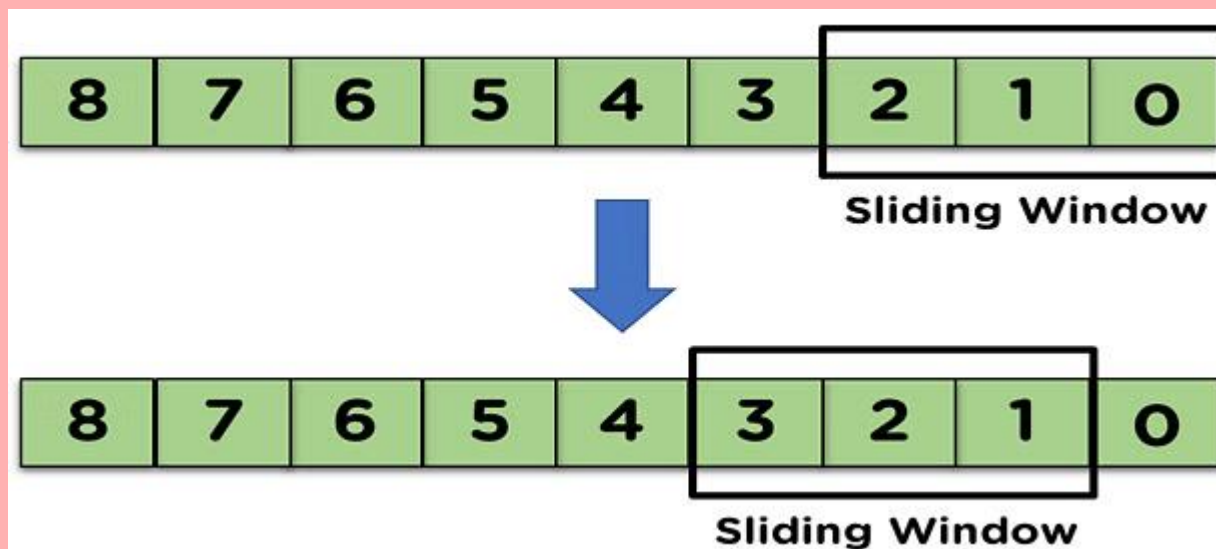


Window Size 3

Figure 1



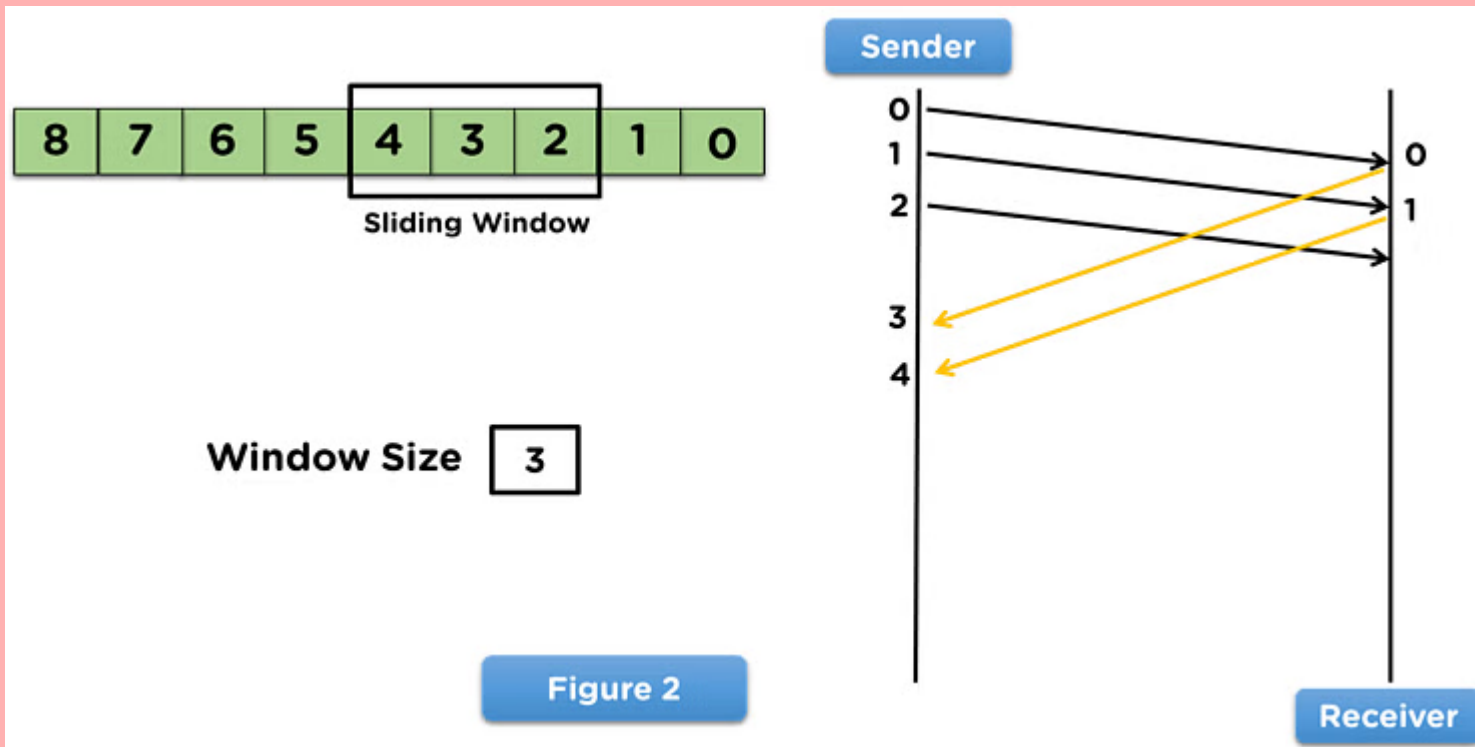
When the receiver transmits the acknowledgment of the first transmitted frame, the sliding window will shift from the acknowledged frame.



Steps for the Receiver Side

- On receiving the data frames from the sender side, the receiver will use the frames in the network model.
- After the receiver uses the frame, it will transmit the acknowledgement to the sender side for that data frame.
- Then, the receiver side will receive the next data frame from the sender side, as mentioned in figure2.

- This process continues until all the frames are transmitted from the sender side to the receiver side, and the receiver side transmits the acknowledgment of all the received frames.



Go-Back-ARQ N

- Go-Back-N Automatic Repeat reQuest (Go-Back-N ARQ), is a data link layer protocol that uses a sliding window method for reliable and sequential delivery of data frames. It is a case of sliding window protocol having to send window size of N and receiving window size of 1.

Working of Go-Back-N ARQ

Suppose there are a sender and a receiver, and let's assume that there are 11 frames to be sent. These frames are represented as 0,1,2,3,4,5,6,7,8,9,10, and these are the sequence numbers of the frames. Mainly, the sequence number is decided by the sender's window size. But, for the better understanding, we took the running sequence numbers, i.e., 0,1,2,3,4,5,6,7,8,9,10. Let's consider the window size as 4,

- **Step 1:** Firstly, the sender will send the first four frames to the receiver, i.e., 0,1,2,3, and now the sender is expected to receive the acknowledgment of the 0th frame.

WORKING OF GO-BACK-N ARQ



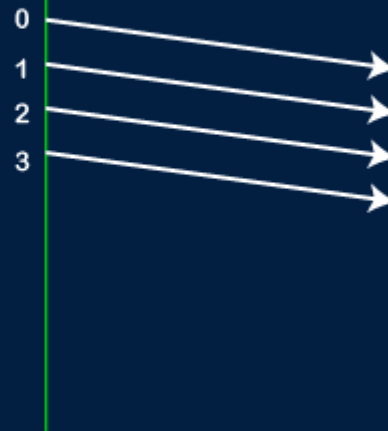
Sliding Window

Window Size:

4

Sender

Receiver



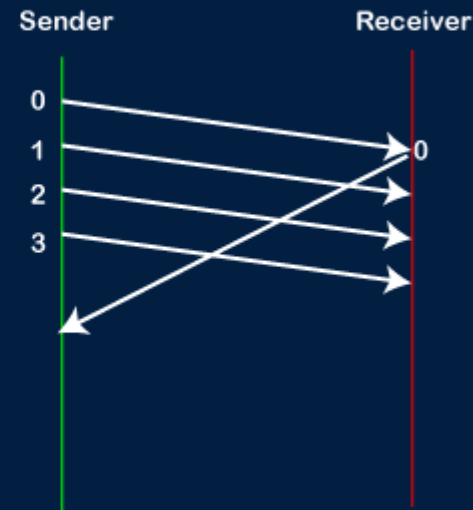
- Let's assume that the receiver has sent the acknowledgment for the 0 frame, and the receiver has successfully received it.

WORKING OF GO-BACK-N ARQ



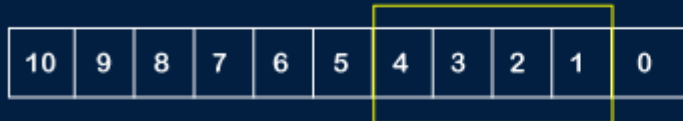
Window Size:

4



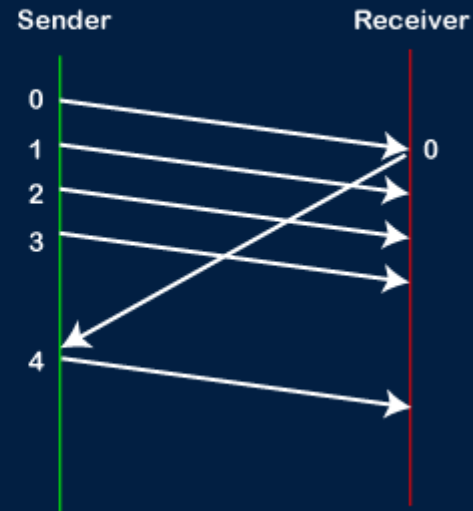
- The sender will then send the next frame, i.e., 4, and the window slides containing four frames (1,2,3,4).

WORKING OF GO-BACK-N ARQ



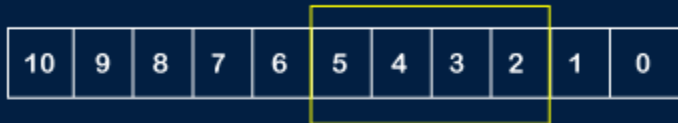
Sliding Window

Window Size: 4



- The receiver will then send the acknowledgment for the frame no 1. After receiving the acknowledgment, the sender will send the next frame, i.e., frame no 5, and the window will slide having four frames (2,3,4,5).

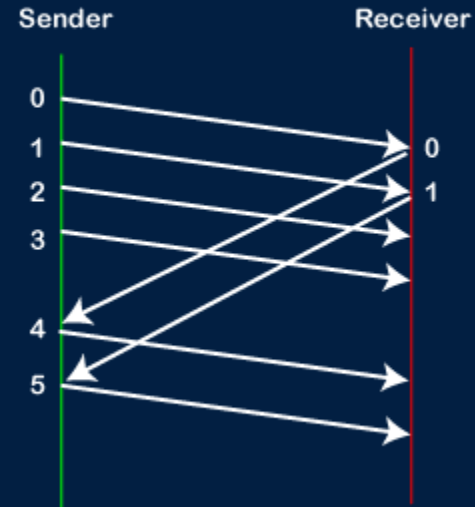
WORKING OF GO-BACK-N ARQ



Sliding Window

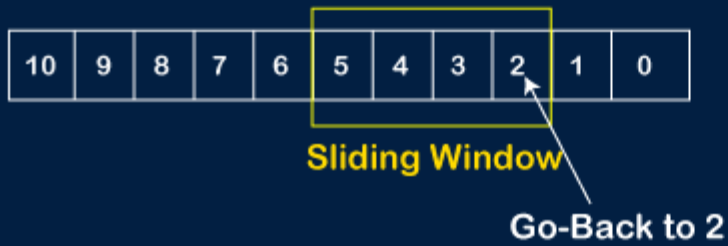
Window Size:

4



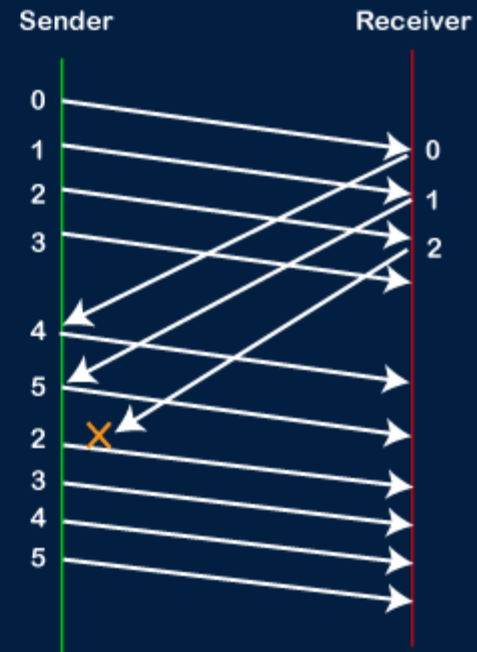
- Now, let's assume that the receiver is not acknowledging the frame no 2, either the frame is lost, or the acknowledgment is lost. Instead of sending the frame no 6, the sender Go-Back to 2, which is the first frame of the current window, retransmits all the frames in the current window, i.e., 2,3,4,5.

WORKING OF GO-BACK-N ARQ



Window Size:

4



- In Go-Back-N, N determines the sender's window size, and the size of the receiver's window is always 1.
- It does not consider the corrupted frames and simply discards them.
- It does not accept the frames which are out of order and discards them.
- If the sender does not receive the acknowledgment, it leads to the retransmission of all the current window frames.

Selective ARQ Protocol

- The selective repeat ARQ is one of the Sliding Window Protocol strategies that is used where reliable in-order delivery of the data packets is required. The selective repeat ARQ is used for noisy channels or links and it manages the flow and error control between the sender and the receiver.
- In the selective repeat ARQ, we only resend the data frames that are damaged or lost. On the other hand, the correct frames are received at the receiver's end and are buffered for future usage. As we only resend the selected damaged frames we name this technique the **Selective Repeat ARQ** technique.
- In the Selective Repeat ARQ protocol, ARQ stands for Automatic Repeat Request. ARQ is an error-control strategy that ensures that a sequence of information is delivered in order and without any errors or duplications despite transmission errors and losses.

Selective Repeat Protocol

- **Working of Selective Repeat Protocol**

- In the selective repeat ARQ, both the sender and the receiver have windows of the same size. The window on the sender's side covers the sequence of data packets that are sent (or to be sent). On the other hand, the window on the receiver's side covers the sequence of data packets that are received (or to be received).

- The size of the sender's window is

$2^{(m-1)}$, where m is the number of bits used in the header of the packet to express the packet's sequence number. The window size of the receiver is the same as that of the sender i.e.

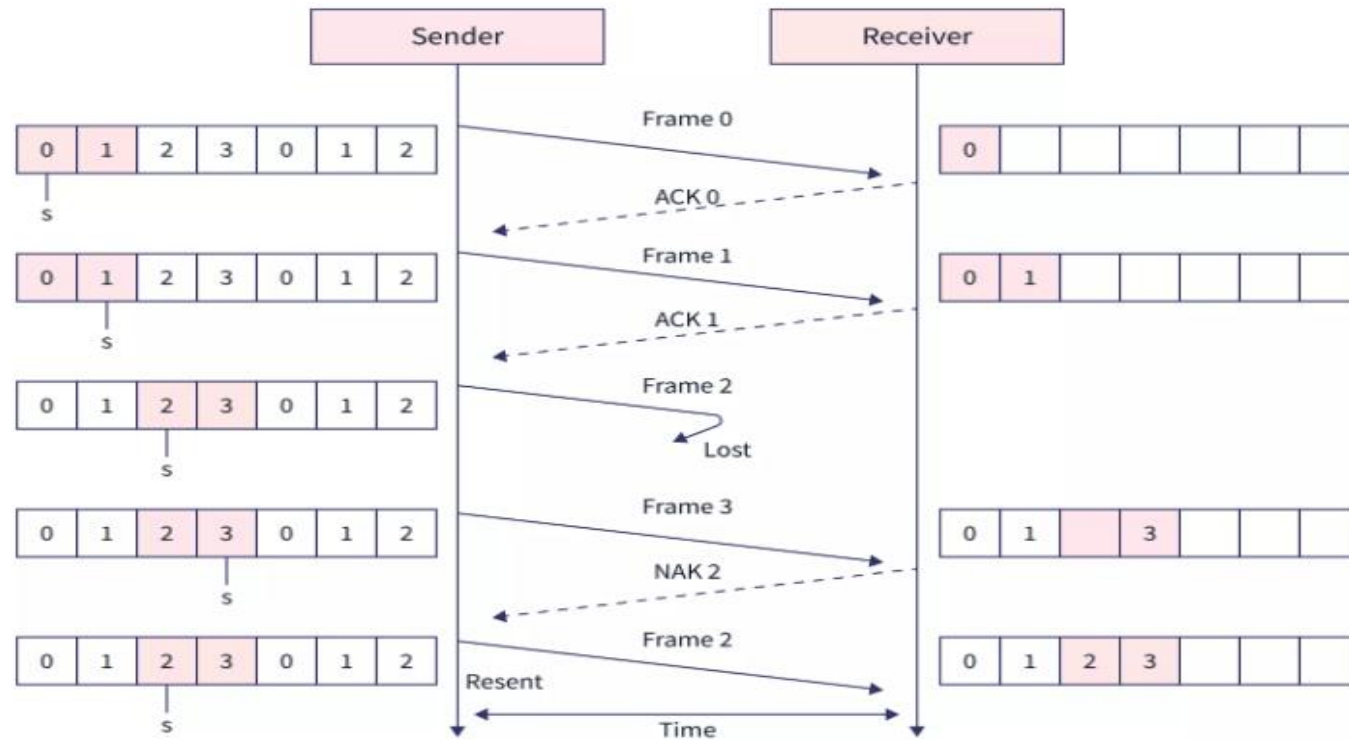
$2^{(m-1)}$.

1. The sender's window size is represented using **Ws**

2. Receiver's window size is represented using **Wr**.

The overall working of the selective repeat ARQ is simple. Initially, the sender sends several frames according to the window size. The receiver on the other end receives the frames and sends the ACK for the correct frames and NACK for lost or damaged frames. The sender re-transmits the frames for which the NACK was sent by the receiver. After receiving all the frames, the receiver sorts the frame according to the sequence number for further usage.

Selective Repeat Protocol



Selective Repeat Protocol

The steps of data transmission can be:

- The sender sends frames 0 and 1.
- The receiver receives the frames and sends back ACK 0 and ACK 1.
- Again the sender sends the frames 2 and 3.
- The receiver only receives the frame 3. So it sends back NACK 2 which means that the 2nd frame is lost and needs to be re-transmitted.
- So, the sender sends back frame 2 and this process is continued till all the frames have been received by the receiver.

HDLC

- **HDLC** (High-Level Data Link Control) is a bit-oriented protocol that is used for communication over the **point-to-point and multipoint links**. This protocol implements the mechanism of ARQ(Automatic Repeat Request). With the help of the HDLC protocol, full-duplex communication is possible.

HDLC

- **HDLC** is the most widely used protocol and offers reliability, efficiency, and a high level of Flexibility.
- In order to make the HDLC protocol applicable for various network configurations, there are three types of stations and these are as follows:
- **Primary Station**
This station mainly looks after data like management. In the case of the communication between the primary and secondary station; it is the responsibility of the primary station to connect and disconnect the data link. The frames issued by the primary station are commonly known as commands.
- **Secondary Station**
The secondary station operates under the control of the primary station. The Frames issued by the secondary stations are commonly known as responses.
- **Combined Station**
The combined station acts as both Primary stations as well as Secondary stations. The combined station issues both commands as well as responses.

HDLC

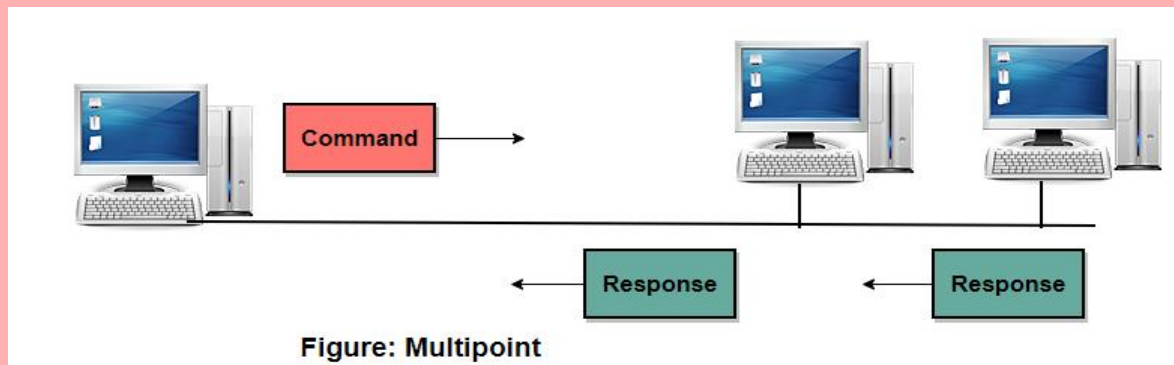
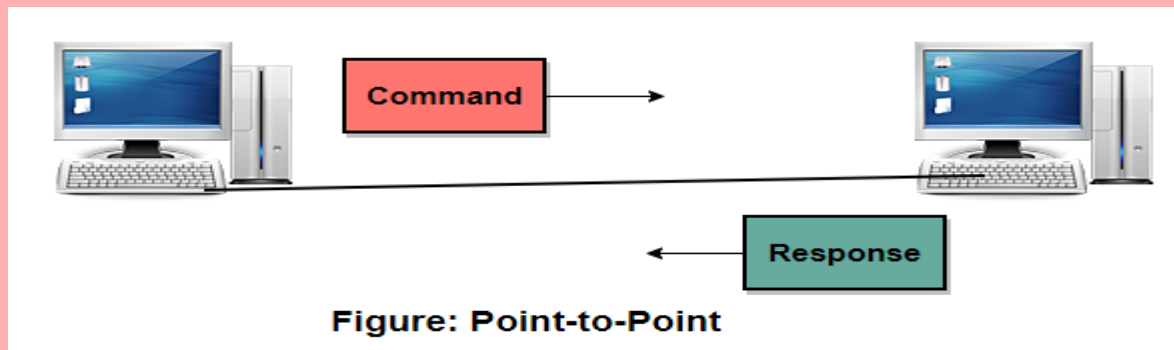
- **Transfer Modes in HDLC**

- The HDLC protocol offers two modes of transfer that mainly can be used in different configurations. These are as follows:
- Normal Response Mode(NRM)
- Asynchronous Balance Mode(ABM)

HDLC

1. Normal Response Mode(NRM)

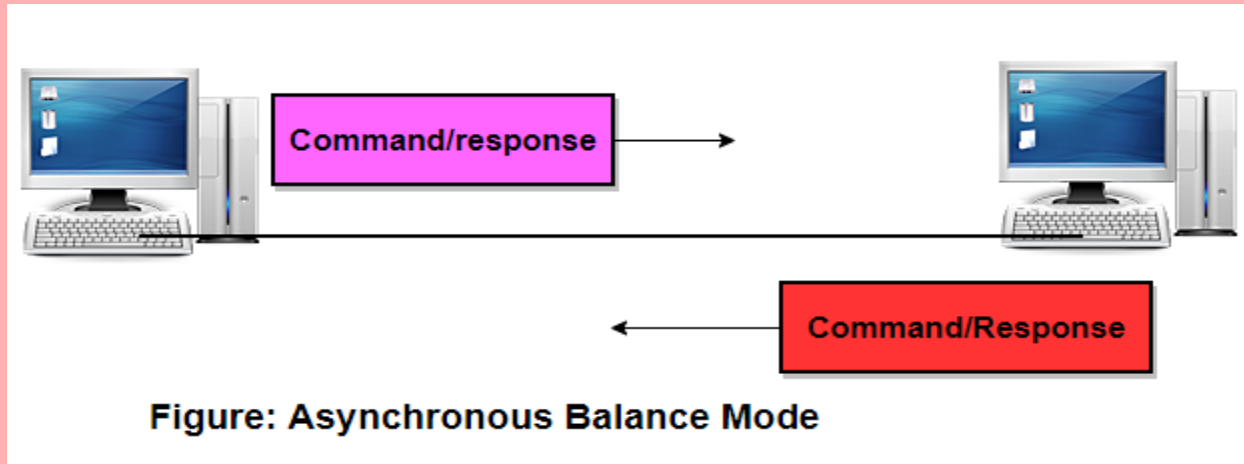
- In this mode, the configuration of the station is unbalanced. There are one primary station and multiple secondary stations. Where the primary station can send the commands and the secondary station can only respond.
- This mode is used for both **point-to-point** as well as **multiple-point links**.



HDLC

2. Asynchronous Balance Mode(ABM)

- In this mode, the configuration of the station is balanced. In this mode, the link is point-to-point, and each station can function as a primary and as secondary.
- Asynchronous Balance mode(ABM) is a commonly used mode today.

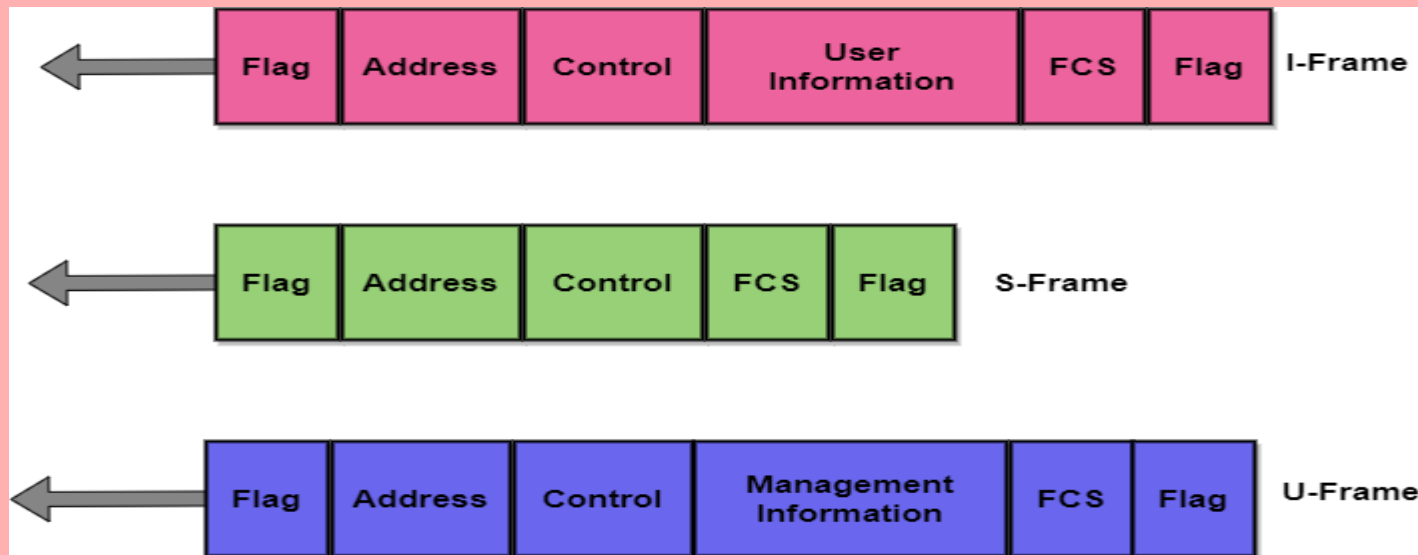


HDLC Frames

- **There are three types of frames defined in the HDLC:**
- **Information Frames(I-frames)**
These frames are used to transport the user data and the control information that is related to the user data. If the first bit of the control field is 0 then it is identified as I-frame.
- **Supervisory Frames(S-frames)**
These frames are only used to transport the control information. If the first two bits of the control field are 1 and 0 then the frame is identified as S-frame
- **Unnumbered Frames(U-Frames)**
These frames are mainly reserved for system management. These frames are used for exchanging control information between the communicating devices.

Frame Format

- There are up to six fields in each HDLC frame. There is a beginning flag field, the address field then, a control field, an information field, a frame check sequence field(FCS), and an ending field.
- In the case of the multiple-frame transmission, the ending flag of the one frame acts as the beginning flag of the next frame.



Frame Format

1. Flag Field

- This field of the HDLC frame is mainly a sequence of 8-bit having the bit pattern 01111110 and it is used to identify the beginning and end of the frame. The flag field mainly serves as a synchronization pattern for the receiver.

2. Address Field

- It is the second field of the HDLC frame and it mainly contains the address of the secondary station. This field can be 1 byte or several bytes long which mainly depends upon the need of the network. In case if the frame is sent by the primary station, then this field contains the address(es) of the secondary stations. If the frame is sent by the secondary station, then this field contains the address of the primary station.

3. Control Field

- This is the third field of the HDLC frame and it is a 1 or 2-byte segment of the frame and is mainly used for flow control and error control. Bits interpretation in this field mainly depends upon the type of the frame.

Frame Format

4. Information Field

- This field of the HDLC frame contains the user's data from the network layer or the management information. The length of this field varies from one network to another.

5. FCS Field

- FCS means Frame check sequence and it is the error detection field in the HDLC protocol. There is a 16 bit CRC code for error detection.

Frame Format

HDLC – FRAME FORMAT



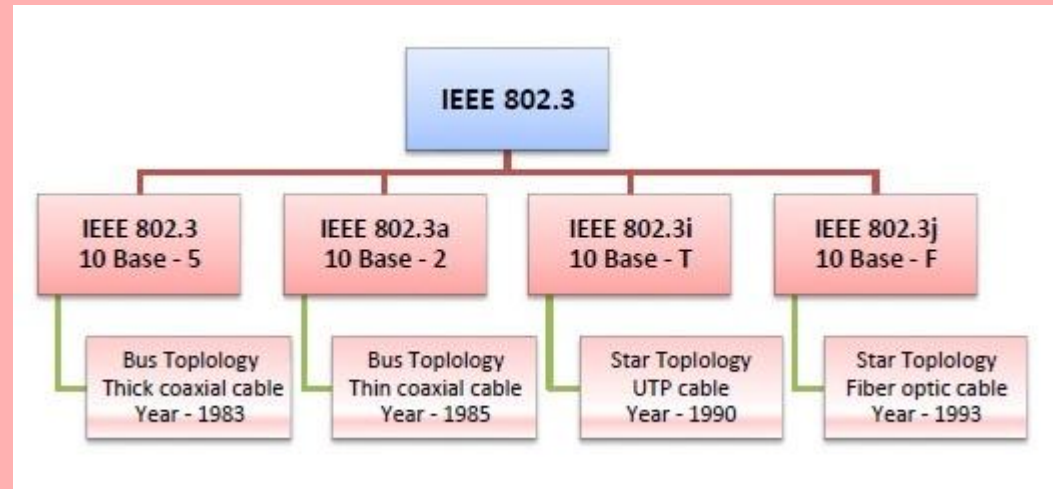
Ethernet

- Ethernet is a set of technologies and protocols that are used primarily in [LANs](#). It was first standardized in 1980s by IEEE 802.3 standard. IEEE 802.3 defines the [physical layer](#) and the [medium access control \(MAC\) sub-layer](#) of the [data link layer](#) for wired Ethernet networks. Ethernet is classified into two categories: classic Ethernet and switched Ethernet.
- Classic Ethernet is the original form of Ethernet that provides data rates between 3 to 10 Mbps. The varieties are commonly referred as 10BASE-X.
- Here,
- 10 is the maximum throughput, i.e. 10 Mbps,
- BASE denoted use of baseband transmission and X is the type of medium used

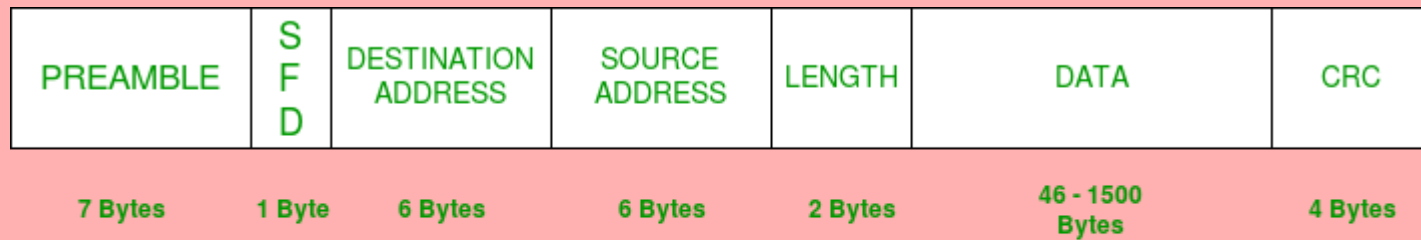
Ethernet

- IEEE 802.3 Popular Versions :-
- **IEEE 802.3:** This was the original standard given for 10BASE-5. It used a thick single coaxial cable into which a connection can be tapped by drilling into the cable to the core. Here, 10 is the maximum throughput, i.e. 10 Mbps, BASE denoted use of baseband transmission, and 5 refers to the maximum segment length of 500m.
- **IEEE 802.3a:** This gave the standard for thin coax (10BASE-2), which is a thinner variety where the segments of coaxial cables are connected by BNC connectors. The 2 refers to the maximum segment length of about 200m (185m to be precise).
- **IEEE 802.3i:** This gave the standard for twisted pair (10BASE-T) that uses unshielded twisted pair (UTP) copper wires as physical layer medium. The further variations were given by IEEE 802.3u for 100BASE-TX, 100BASE-T4 and 100BASE-FX.
- **IEEE 802.3j:** This gave the standard for Ethernet over Fiber (10BASE-F) that uses fiber optic cables as medium of transmission.

Ethernet



Ethernet (IEEE 802.3) Frame Format



IEEE 802.3 ETHERNET Frame Format

Ethernet (IEEE 802.3) Frame Format

- **PREAMBLE:** Ethernet frame starts with a 7-Bytes Preamble. This is a pattern of alternative 0's and 1's which indicates starting of the frame and allow sender and receiver to establish bit synchronization. Initially, PRE (Preamble) was introduced to allow for the loss of a few bits due to signal delays. But today's high-speed Ethernet doesn't need a Preamble to protect the frame bits. PRE (Preamble) indicates the receiver that frame is coming and allow the receiver to lock onto the data stream before the actual frame begins.
- **Start of frame delimiter (SFD):** This is a 1-Byte field that is always set to 10101011. SFD indicates that upcoming bits are starting the frame, which is the destination address. Sometimes SFD is considered part of PRE, this is the reason Preamble is described as 8 Bytes in many places. The SFD warns station or stations that this is the last chance for synchronization.
- **Destination Address:** This is a 6-Byte field that contains the MAC address of the machine for which data is destined.
- **Source Address:** This is a 6-Byte field that contains the MAC address of the source machine. As Source Address is always an individual address (Unicast), the least significant bit of the first byte is always 0.
- **Length:** Length is a 2-Byte field, which indicates the length of the entire Ethernet frame. This 16-bit field can hold a length value between 0 to 65535, but length cannot be larger than 1500 Bytes because of some own limitations of Ethernet.

Ethernet (IEEE 802.3) Frame Format

- **Data:** This is the place where actual data is inserted, also known as **Payload**. Both [IP header](#) and data will be inserted here if Internet Protocol is used over Ethernet. The maximum data present may be as long as 1500 Bytes. In case data length is less than minimum length i.e. 46 bytes, then padding 0's is added to meet the minimum possible length.
- **Cyclic Redundancy Check (CRC):** CRC is 4 Byte field. This field contains a 32-bits hash code of data, which is generated over the Destination Address, Source Address, Length, and Data field. If the checksum computed by destination is not the same as sent checksum value, data received is corrupted.
- **VLAN Tagging:** The Ethernet frame can also include a VLAN (Virtual Local Area Network) tag, which is a 4-byte field inserted after the source address and before the EtherType field. This tag allows network administrators to logically separate a physical network into multiple virtual networks, each with its own VLAN ID.
- **Jumbo Frames:** In addition to the standard Ethernet frame size of 1518 bytes, some network devices support Jumbo Frames, which are frames with a payload larger than 1500 bytes. Jumbo Frames can increase network throughput by reducing the overhead associated with transmitting a large number of small frames.
- **Ether Type Field:** The EtherType field in the Ethernet frame header identifies the protocol carried in the payload of the frame. For example, a value of 0x0800 indicates that the payload is an IP packet, while a value of 0x0806 indicates that the payload is an ARP (Address Resolution Protocol) packet.
- **Multicast and Broadcast Frames:** In addition to Unicast frames (which are sent to a specific destination MAC address), Ethernet also supports Multicast and Broadcast frames. Multicast frames are sent to a specific group of devices that have joined a multicast group, while Broadcast frames are sent to all devices on the network.
- **Collision Detection:** In half-duplex Ethernet networks, collisions can occur when two devices attempt to transmit data at the same time. To detect collisions, Ethernet uses a [Carrier Sense Multiple Access with Collision Detection \(CSMA/CD\)](#) protocol, which listens for activity on the network before transmitting data and backs off if a collision is detected.

Ethernet (IEEE 802.3) Frame Format

Advantages

- **Simple Format:** The Ethernet frame format is simple and easy to understand, making it easy to implement and troubleshoot Ethernet networks.
- **Flexibility:** The Ethernet frame format is flexible and can accommodate different data sizes and network topologies, making it suitable for a wide range of network applications.
- **Widely Adopted:** The Ethernet frame format is widely adopted and supported by a large number of vendors and network devices, ensuring compatibility and interoperability.
- **Error Detection:** The Ethernet frame format includes a [cyclic redundancy check \(CRC\)](#) field for error detection, which helps to ensure data integrity during transmission.
- **Support for VLANs:** The Ethernet frame format supports virtual local area networks (VLANs), which allows network administrators to logically partition a physical LAN into multiple smaller virtual LANs for improved network management and security.

Disadvantages

- **Limited Frame Size:** The Ethernet frame format has a maximum frame size of 1500 bytes, which can limit the amount of data that can be transmitted in a single frame and can result in increased overhead due to [fragmentation](#) and reassembly of larger packets.
- **Broadcast Storms:** Ethernet networks use broadcast transmissions to send frames to all devices on the network, which can lead to broadcast storms if too many devices send broadcast frames simultaneously, resulting in network congestion and performance issues.
- **Security Vulnerabilities:** The Ethernet frame format does not include built-in security features, making Ethernet networks vulnerable to security threats such as eavesdropping and [spoofing](#).
- **Limited Speed:** Ethernet networks have a limited maximum speed, which may not be sufficient for high-speed applications or large-scale networks.
- **Limited Distance:** The maximum distance between two devices on an Ethernet network is limited, which can restrict the physical coverage of the network.